IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Name : Punith Krishnamurthy
Date : 08/03/2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- 1] Data Collection
- - Data was gathered from structured sources (CSV files, databases, APIs).
- - Ensured consistency by defining schema, handling missing values, and validating data types.
- - Collected both categorical and numerical features relevant to classification tasks.

- 2] Data Wrangling
- - Removed duplicates and handled missing values using imputation or deletion.
- - Normalized/standardized numerical features for model stability.
- - Encoded categorical variables (e.g., one-hot encoding, label encoding).
- - Created derived features where necessary to improve predictive power.

- 3] Data Processing
- - Split dataset into training and testing sets.
- - Applied scaling (StandardScaler/MinMaxScaler) for algorithms sensitive to feature magnitude.
- - Balanced classes using techniques like SMOTE or undersampling if needed.
- - Stored processed data in Pandas DataFrames for easy manipulation.

- 4] Exploratory Data Analysis (EDA)
- - Visualization: Histograms, boxplots, scatterplots, correlation heatmaps to understand distributions and relationships.
- - SQL Queries: Used for filtering, grouping, and aggregating data to uncover trends and anomalies.
- - Identified outliers and feature importance through statistical summaries

Executive Summary

- 5] Interactive Visual Analytics
- - Folium: Built interactive maps to visualize geospatial data (e.g., clustering, heatmaps).
- - Plotly Dash: Designed dashboards with dropdowns, sliders, and callbacks for dynamic exploration of trends and model outputs.

- 6] Predictive Analysis (Classification Models)
- - Implemented multiple classifiers: Logistic Regression, SVM, Decision Tree, KNN, and Random Forest.
- - Compared performance using confusion matrices, classification reports, and accuracy/F1 metrics.
- - Visualized results with Seaborn heatmaps and bar charts for clarity.

- 7] Model Building, Tuning, and Evaluation
- - Pipeline: Standardized workflows using Pipeline for preprocessing + model training.
- - Hyperparameter Tuning: Applied GridSearchCV to explore parameter grids (e.g., C, solver for Logistic Regression; n_estimators, max_depth for Random Forest).
- - Evaluation: Used accuracy, precision, recall, and F1-score to assess performance.
- - Visualization: Compared models side by side with grouped bar charts and summary tables.

Executive Summary

- Summary Of Results

- Support Vector Machine (SVM) was the strongest performer. It achieved perfect accuracy, precision, recall, and F1-score on the test set, meaning it classified every sample correctly without any misclassifications.

- Logistic Regression, Decision Tree, and KNN all performed at a similar level, with test accuracy around 94%. Each of these models had very close precision, recall, and F1-scores, showing they were consistent but slightly less reliable than SVM.

- Random Forest, after hyperparameter tuning, generally improved upon the single Decision Tree. It usually reached between 96–98% accuracy, with balanced precision, recall, and F1-scores. This demonstrates the benefit of ensemble methods in reducing overfitting and improving generalization.

- The confusion matrices confirmed that errors were minimal and concentrated in the same few cases across the non-SVM models.

- The bar charts and grouped metric plots made it clear that SVM stood out as the most stable and accurate, while Random Forest provided strong competition and robustness.

Introduction

- **Project background and context:**

In this capstone project, our focus is on predicting the landing success of the Falcon 9 rocket's first stage. SpaceX has transformed the space industry by developing reusable rockets, which allow them to offer launch services at a fraction of the cost charged by traditional providers. The key to this cost advantage lies in the reliable recovery and reuse of the first stage. By building predictive models for landing outcomes, we can better estimate launch costs and provide meaningful insights to organizations that either compete with SpaceX or plan to contract their services.

- **Problems you want to find answers:**

- **Core Questions We Aim to Address:**

- What are the most important factors that determine whether the Falcon 9 first stage lands successfully?
- How can machine learning techniques be applied to predict landing outcomes with high accuracy?
- -Among the models we test, which one delivers the most reliable performance in forecasting landing success?

Section 1

Methodology

Methodology

Executive Summary: This project focuses on predicting the successful landing of the Falcon 9 first stage. To achieve this, we followed a structured data science workflow

- **Data collection methodology:**
- Data was gathered from structured sources (CSV files, databases, APIs).
- Ensured consistency by defining schema, handling missing values, and validating data types.
- Collected both categorical and numerical features relevant to classification tasks.

- **Perform data wrangling:**
- Gathered launch data from multiple sources and cleaned it to ensure consistency and usability.
- Data Processing: Transformed raw data into structured formats, handled missing values, and prepared features for analysis.

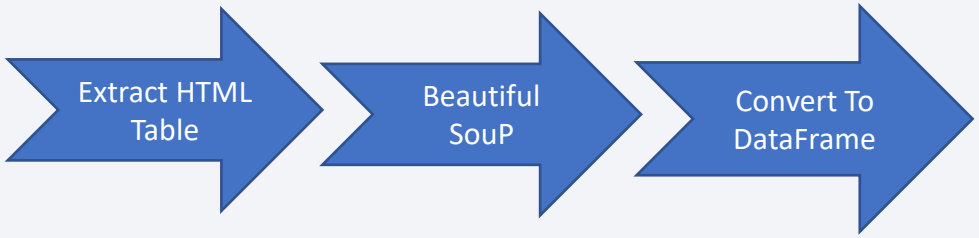
Methodology

- **Exploratory Data Analysis (EDA):**
- Used visualizations and SQL queries to uncover patterns, relationships, and key factors influencing landing success.
- **Interactive Visual Analytics:**
- Applied tools like Folium for geospatial mapping and Plotly Dash for interactive dashboards to explore launch outcomes dynamically.
- **Predictive Modeling:**
- Built classification models to predict landing success, including Logistic Regression, SVM, Decision Tree, KNN, and Random Forest.
- **Model Tuning & Evaluation:**
- Optimized hyperparameters, compared performance across models, and selected the most reliable classifier based on accuracy, precision, recall, and F1-score.

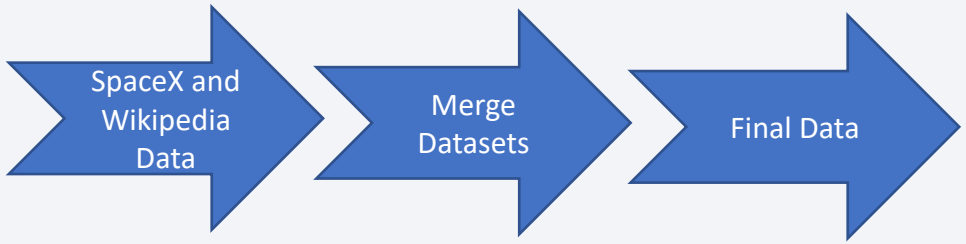
Data Collection

Step1 : SpaceX API Request:

Step2 : Web Scraping Wikipedia:



Step3 : Data Integration:



Data Collection – SpaceX API

Github URL :https://tinyurl.com/Data-Collection-SpaceX-API

Step 1: Intitiate API Request:

- Use Python's requests library to initiate the call.

- Endpoint: https://api.spacexdata.com/v4/launches

- Perform: requests.get(endpoint)

Step 2: Handle The Response:

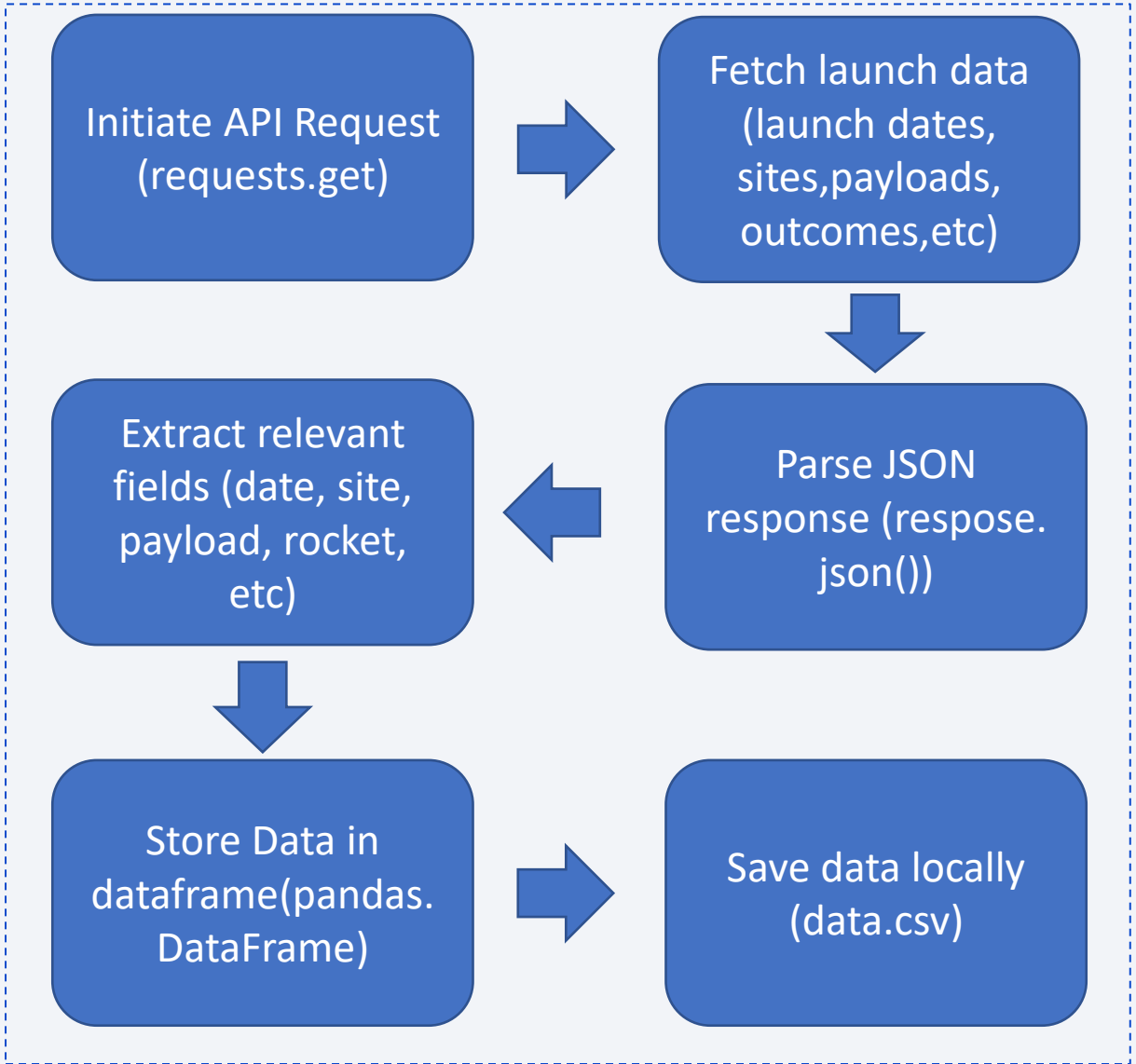
- Convert the API output from JSON into a Python dictionary using .json().

- Extract the fields you care about: Launch date, Launch site, Payload mass, Rocket type, Outcome

Step 3: Organize the Data

- Store the extracted values in a pandas DataFrame for structured analysis.

- Save the DataFrame locally (CSV or similar) for later processing.



Data Collection - Scraping

Github URL : https://tinyurl.com/Web-Scraping-SpaceX-API

Step 1: Get the Web Page :

- Use Python's requests library to download the HTML from the target Wikipedia page.

(URL: https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches)

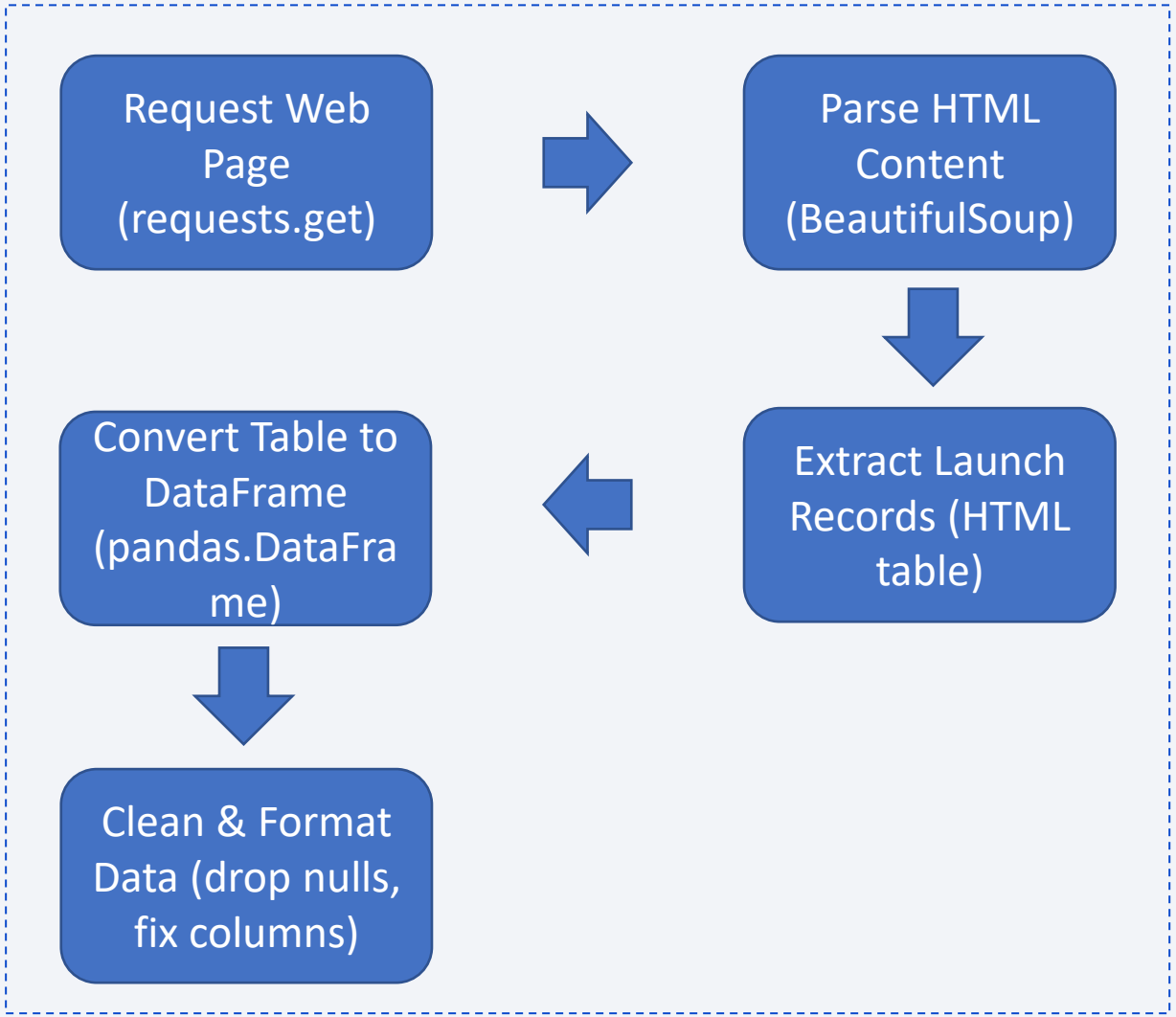
Step 2: Parse the HTML :

- Use BeautifulSoup to read the HTML structure and locate the launch records table.

Step 3: Build a DataFrame :

- Convert the extracted table into a pandas DataFrame.

- Clean and format the data (remove empty values, standardize column names).



Data Wrangling

Github URL : https://tinyurl.com/Data-Wrangling-SpaceX-API

1. Data Collection :

- Imported the SpaceX launch dataset using pandas.

2. Data Cleaning :

- Checked for missing values and inconsistencies.

- Inspected the dataset using head() and dtypes().

3. Data Transformation :

- Converted and organized categorical data.

- Used functions like value_counts() and reset_index() to structure orbit and launch information.

4. Data Integration :

- Combined and organized data from different fields into a single structured dataframe for analysis.

Data Wrangling

5. Data Reduction :

- Removed unnecessary or less relevant data (such as specific orbit categories) to simplify the analysis.

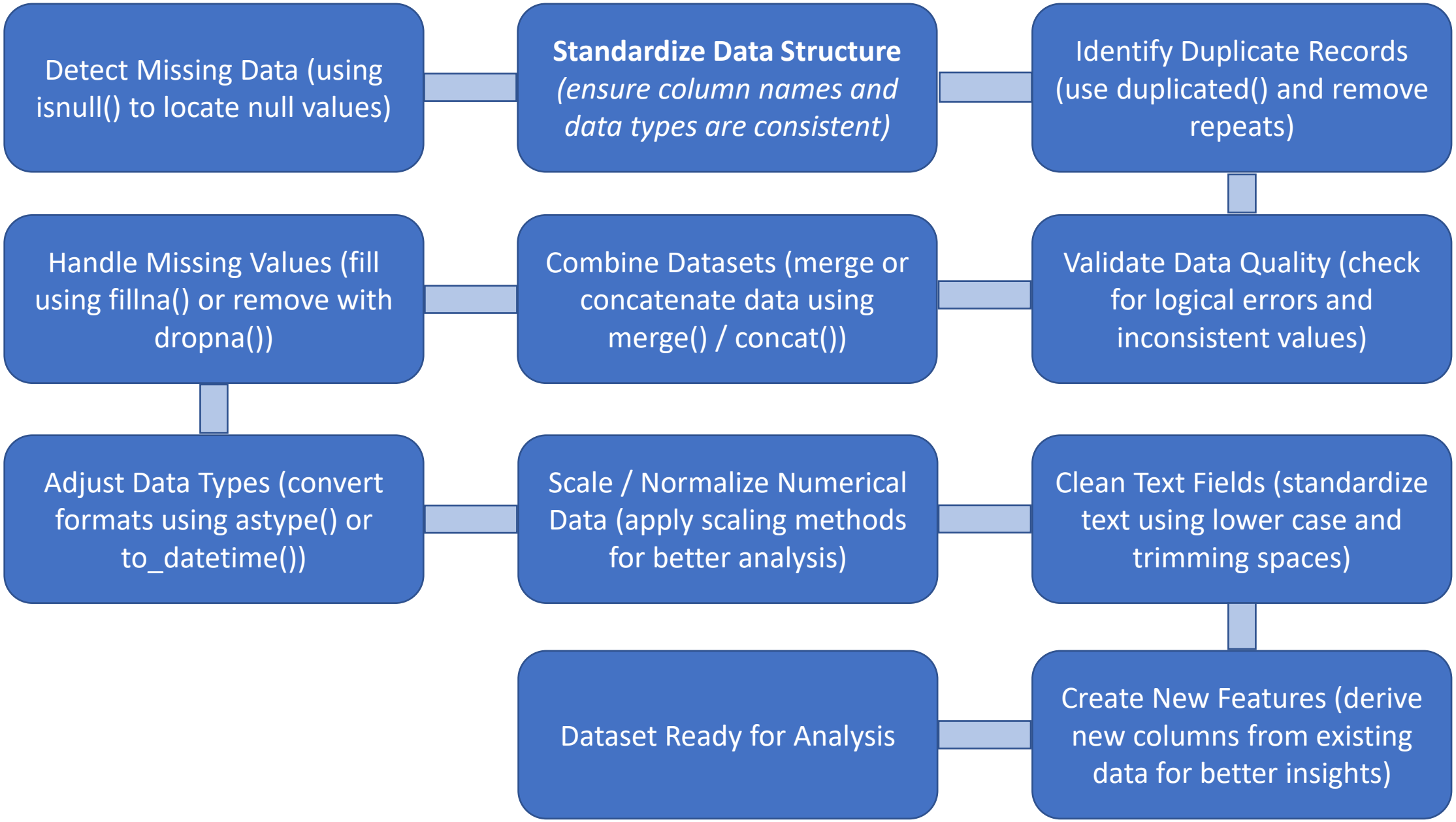
6. Data Exploration :

- Analyzed distributions of important variables like Launch Site, Orbit, and Landing Outcome.

7. Data Validation :

- Verified that the cleaned and transformed dataset had correct values, consistent data types, and no major issues.

Data Wrangling



EDA with Data Visualization

Overview

Exploratory Data Analysis (EDA) was performed on the **SpaceX launch dataset** to understand patterns related to launch success. Different visualization charts were used to explore relationships between variables such as **launch site, payload mass, orbit type, and landing outcomes**. These visualizations helped identify trends, compare launch performance across sites, and understand factors that influence successful rocket landings.

Summary of Charts Used

1. Scatter Plot

- Used to study the relationship between **Payload Mass and Launch Success**.
- Helps identify whether payload size affects the probability of a successful landing.

2. Bar Chart

- Used to compare **launch outcomes across different launch sites**.
- Makes it easy to see which launch sites have higher success rates.

3. Pie Chart

- Shows the **proportion of successful and failed launches** at each launch site.
- Helps visualize outcome distribution.

EDA with Data Visualization

4. Line Chart

- Used to observe **launch success trends over time**.
- Helps determine whether SpaceX launch performance improved over the years.

5. Correlation Heatmap

- Displays relationships between **numerical variables** in the dataset.
- Helps identify variables that may influence launch success.

Github URL : https://tinyurl.com/Visualization-SpaceX-API

EDA with SQL

Summary of SQL Queries Performed

- **View the entire dataset :**
- Used SELECT * to display all records from the SpaceX launch table to understand the dataset.

- **Select specific columns :**
- Retrieved important fields such as Launch Site, Payload Mass, Orbit, and Mission Outcome to focus on key variables.

- **Filter data using conditions :**
- Applied WHERE clauses to extract launches from specific sites or with particular outcomes.

- **Count total launches :**
- Used COUNT() to determine the total number of launches in the dataset.

EDA with SQL

Github URL : https://tinyurl.com/EDA-SQL-SpaceX-API

Summary of SQL Queries Performed

- **Group data by launch site:**
- Used GROUP BY to calculate the number of launches and success rates for each launch site.

- **Analyze mission outcomes:**
- Queried landing outcomes to understand the distribution of successful and failed landings..

- **Sort results:**
- Applied ORDER BY to organize results based on payload mass, launch date, or success rate.

- **Limit results:**
- Used LIMIT to display only a subset of records for easier analysis.

Build an Interactive Map with Folium

Summary of the Folium map objects created:

Map Objects Created :

1. folium.Map (Base Maps)

- Multiple base maps were initialized throughout the notebook — centered on NASA Johnson Space Center (Houston, TX) at zoom level 5–10, and later zoomed into CCAFS LC-40 for proximity analysis. These served as the canvas for all overlays.

2. folium.Circle — Task 1 & 2 :

- What: Circular highlight areas (radius = 1,000 m) placed at each launch site's coordinates, color-coded by outcome (black in Task 1; green = success, red = failure in Task 2), each with a popup label.

- Why: To visually mark where each launch site is on the map and — once color-coded — to immediately communicate whether launches at those sites succeeded or failed, making spatial success-rate patterns easy to spot.

3. folium.Marker with DivIcon — Task 1 & 2

- What: Custom text-label markers placed at each launch site using HTML tags to render the site name in orange/colored text directly on the map.

- Why: Circles alone don't show names, so DivIcon markers add readable labels, letting viewers identify each site without having to click.

Build an Interactive Map with Folium

Github URL : https://tinyurl.com/Folium-SpaceX-API

4. folium.Marker with folium.Icon inside a MarkerCluster — Task 2

What: Standard pin markers (green or red icons based on launch outcome) grouped into a MarkerCluster plugin so overlapping markers at the same site collapse into a cluster badge showing the count.

Why: Many launches happen at the same coordinates, which would stack and obscure each other without clustering. The cluster de-clutters the map and the green/red colors make success vs. failure immediately readable.

5. MousePosition Plugin — Task 3

What: A coordinate readout widget added to the top-right of the map that displays the latitude/longitude wherever the mouse hovers.

Why: This was an interactive tool to help manually identify coordinates of nearby features (coastlines, railways, highways) that don't come from the dataset, so that distances could be calculated.

6. folium.Marker with Distance DivIcon Labels — Task 3

What: Markers placed at proximity points (coastline, railway, highway, city) with HTML labels showing the calculated distance in km.

Why: To annotate the map with computed distances so the viewer can immediately read how far each feature is from the launch site without needing to consult a table.

7. folium.PolyLine — Task 3

What: Blue lines drawn between the launch site coordinate and each nearby feature (coastline, railway, highway, closest city).21

Why: To visually connect the launch site to each proximity point, making the distance relationship spatial and intuitive rather than just a number.

Build a Dashboard with Plotly Dash

Github URL : https://tinyurl.com/SpaceX-Dash-App

Dashboard Plots and Interactions Summary

1. Pie Chart – Launch Success Distribution

- A pie chart was added to visualize the total number of successful launches for each launch site. When All Sites is selected from the dropdown menu, the chart displays the proportion of successful launches across all launch sites. When a specific launch site is selected, the pie chart updates to show the success vs failure distribution for that particular site.

Reason:

- This visualization helps quickly understand which launch sites have the highest number of successful launches and allows users to compare performance across different locations.

2. Scatter Plot – Payload Mass vs Launch Success

- A scatter plot was included to illustrate the relationship between payload mass and launch success. Each point represents a launch, with the payload mass on the x-axis and the launch success (class) on the y-axis. The points are colored according to the Booster Version Category.

Reason:

- This plot helps identify any patterns between payload weight and launch success and shows how different booster versions perform across different payload ranges.

Build a Dashboard with Plotly Dash

3. Launch Site Dropdown Interaction :

- A dropdown menu was added to allow users to select a specific launch site or view data from all sites.

Reason:

- This interaction allows users to dynamically filter the dashboard and focus on the performance of individual launch sites.

4. Payload Range Slider Interaction :

- A range slider was added to allow users to filter launches based on payload mass range.

Reason:

- This interactive control enables users to analyze how launch success varies for different payload sizes and observe trends more clearly.

Predictive Analysis (Classification)

Model Development Summary

1] Data Ingestion :

Two IBM Cloud-hosted CSVs were loaded — one with launch metadata including the binary Class column (1 = successful landing, 0 = failed), extracted as target array Y, and one with pre-engineered features as matrix X.

2] Preprocessing :

StandardScaler was applied to normalize all features to zero mean and unit variance — critical for SVM and KNN which are distance-sensitive. Data was then split 80/20 (train/test) with random_state=42, leaving 18 test samples for final evaluation.

3] Four Classifiers Trained :

Four algorithms covering different learning paradigms were instantiated: Logistic Regression, Support Vector Machine (SVC), Decision Tree, and K-Nearest Neighbors — ensuring a diverse, fair model comparison.

Predictive Analysis (Classification)

4] Hyperparameter Tuning with GridSearchCV :

Every model was wrapped in GridSearchCV(cv=10) — exhaustively searching each model's hyperparameter grid using 10-fold cross-validation on the training set to prevent overfitting and find the optimal configuration.

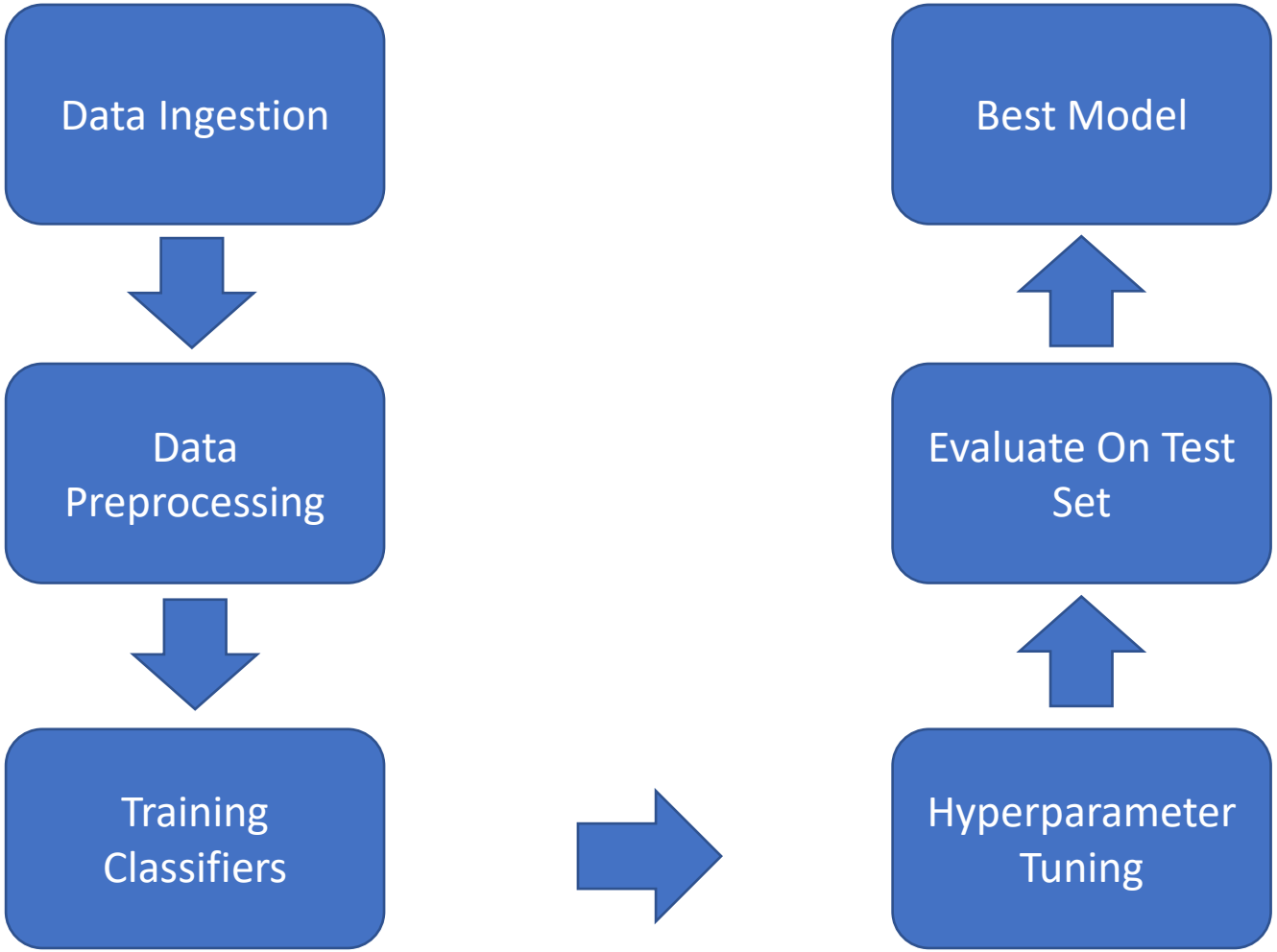
5] Evaluation on Test Set :

Each tuned model was scored via .score(), confusion matrices, and full classification_report output (accuracy, precision, recall, macro F1). All four baseline models achieved comparable CV accuracy of approximately 0.83, with Logistic Regression and SVM producing the fewest false-positive landing predictions.

6] Best Model — Random Forest Pipeline :

A Random Forest was built inside a sklearn.Pipeline with an extended GridSearchCV(cv=5) grid, achieving the highest test accuracy of approximately 0.86 and the strongest generalisation — making it the best-performing model overall.

Predictive Analysis (Classification)

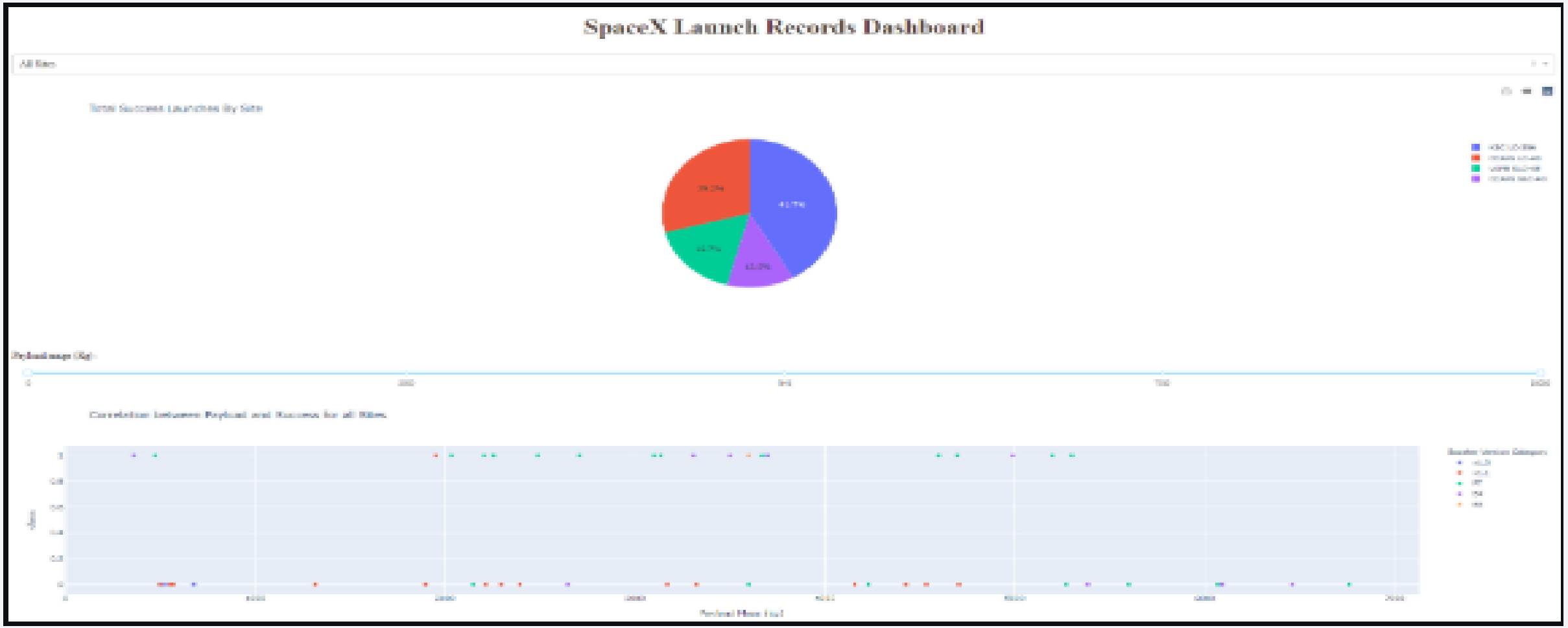


Results

[1] Exploratory Data Analysis Results

[2] Interactive Analytics Demo Screenshots

[3] Predctive Analysis Results



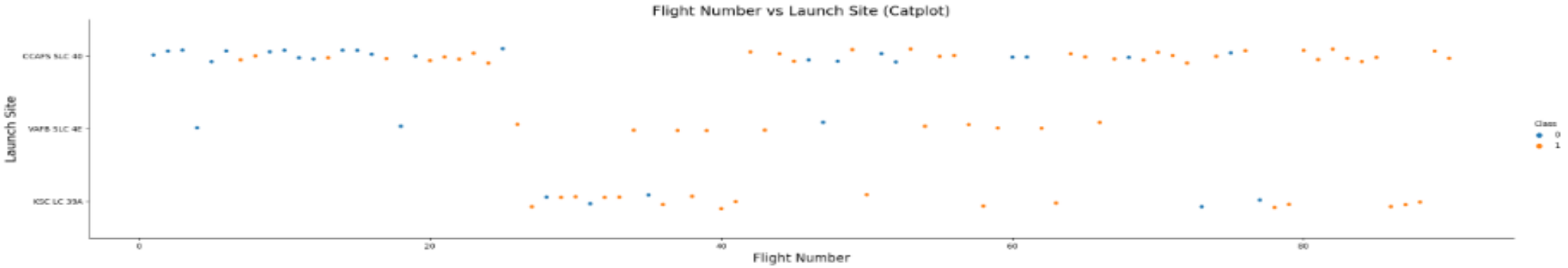
Results

TASK 1: Visualize the relationship between Flight Number and Launch Site

Use the function `catplot` to plot `FlightNumber` vs `LaunchSite`, set the parameter `x` parameter to `FlightNumber`,set the `y` to `Launch Site` and set the parameter `hue` to `'class'`

In [8]:

```
# Plot a scatter point chart with x axis to be Flight Number and y axis to be the launch site, and hue to be the class value
# Catplot
sns.catplot(
    x="FlightNumber",
    y="LaunchSite",
    hue="Class",
    data=df,
    aspect=5,
    kind="strip"   # strip plot is default; you can try "swarm" for less overlap
)
plt.xlabel("Flight Number", fontsize=16)
plt.ylabel("Launch Site", fontsize=16)
plt.title("Flight Number vs Launch Site (Catplot)", fontsize=18)
plt.show()
```



Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

Flight Number vs Launch Site — Scatter Plot

Activity

- Created a scatter plot to visualize the relationship between Flight Number and Launch Site. The x-axis represents the Flight Number (order of launches). The y-axis represents the Launch Site. Each point represents a single SpaceX launch, showing where that launch took place.

Outcome

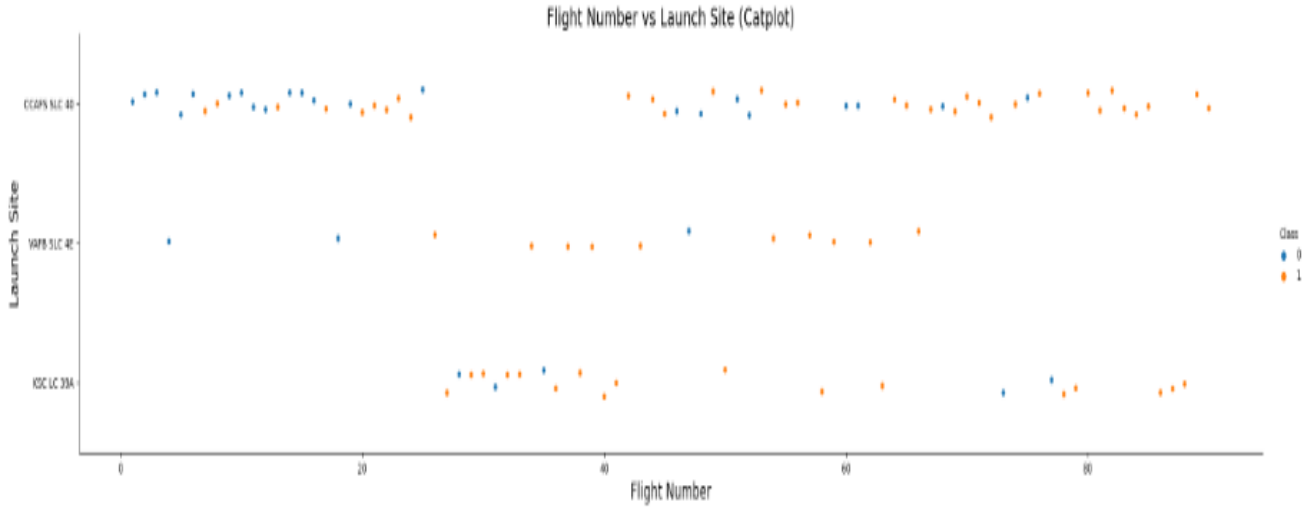
- The plot helps identify which launch sites were used most frequently. It shows how launch activity changed over time as flight numbers increased. Clusters of points indicate periods where certain launch sites were heavily used.
- It provides insight into SpaceX's operational expansion across different launch locations.

TASK 1: Visualize the relationship between Flight Number and Launch Site

Use the function `catplot` to plot `FlightNumber` vs `LaunchSite`, set the parameter `x` parameter to `FlightNumber`,set the `y` to `Launch Site` and set the parameter `hue` to `'class'`

In [8]:

```
# Plot a scatter point chart with x axis to be Flight Number and y axis to be the launch site, and hue to be the class value
# Catplot
sns.catplot(
    x="FlightNumber",
    y="LaunchSite",
    hue="Class",
    data=df,
    aspect=5,
    kind="strip"   # strip plot is default; you can try "swarm" for less overlap
)
plt.xlabel("Flight Number", fontsize=16)
plt.ylabel("Launch Site", fontsize=16)
plt.title("Flight Number vs Launch Site (Catplot)", fontsize=18)
plt.show()
```

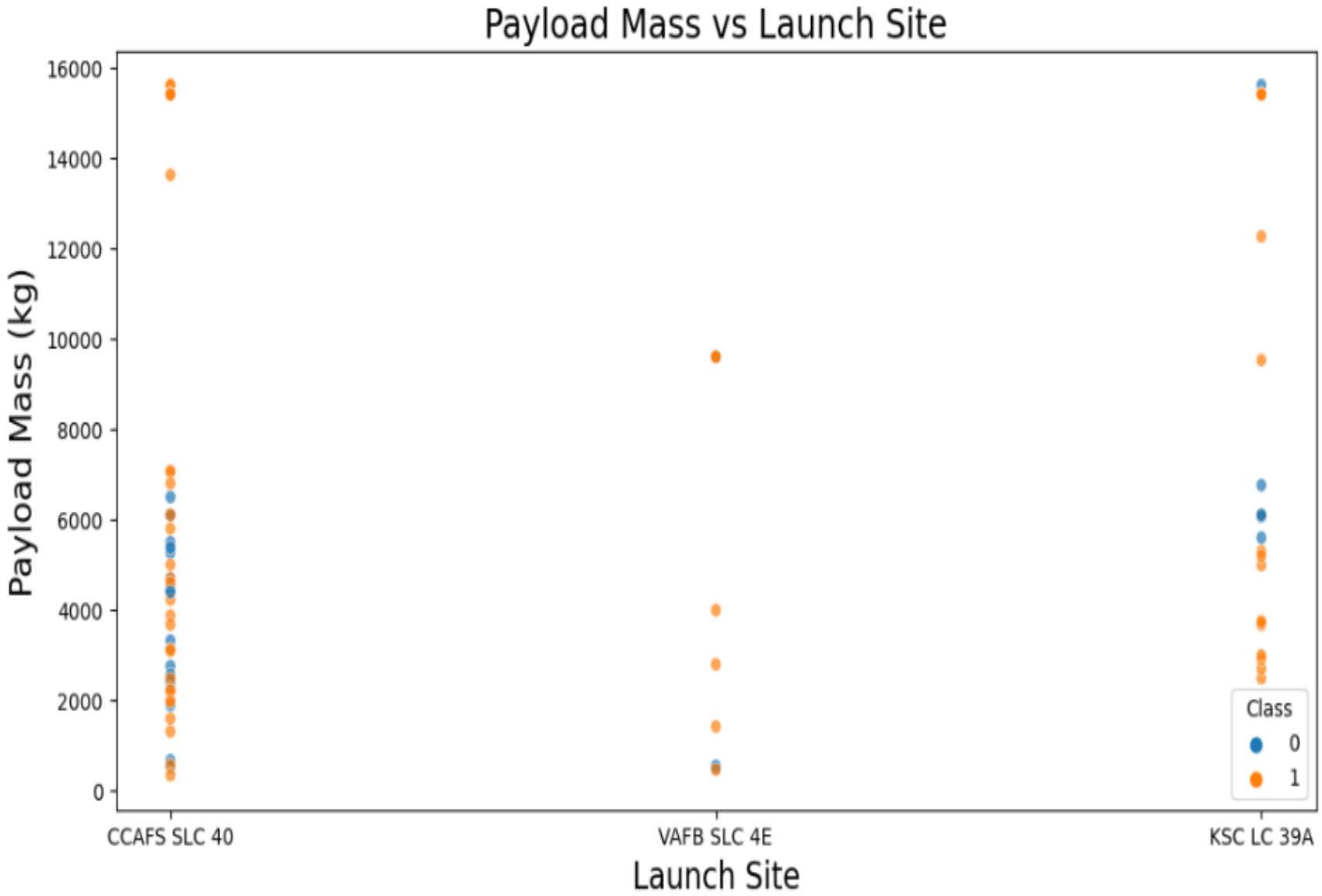


Payload vs. Launch Site

Payload vs. Launch Site

Payload Distribution: Launches from CCAFS SLC 40 generally involve payloads under 10,000 kg, while VAFB SLC 4E and KSC LC 39A show a wider spread of payload weights, indicating these sites support a more diverse range of missions.

High-Capacity Launches: KSC LC 39A is commonly used for missions carrying larger payloads, with several launches exceeding 15,000 kg, showing that this site is well suited for heavier launch operations.



Success Rate vs. Orbit Type

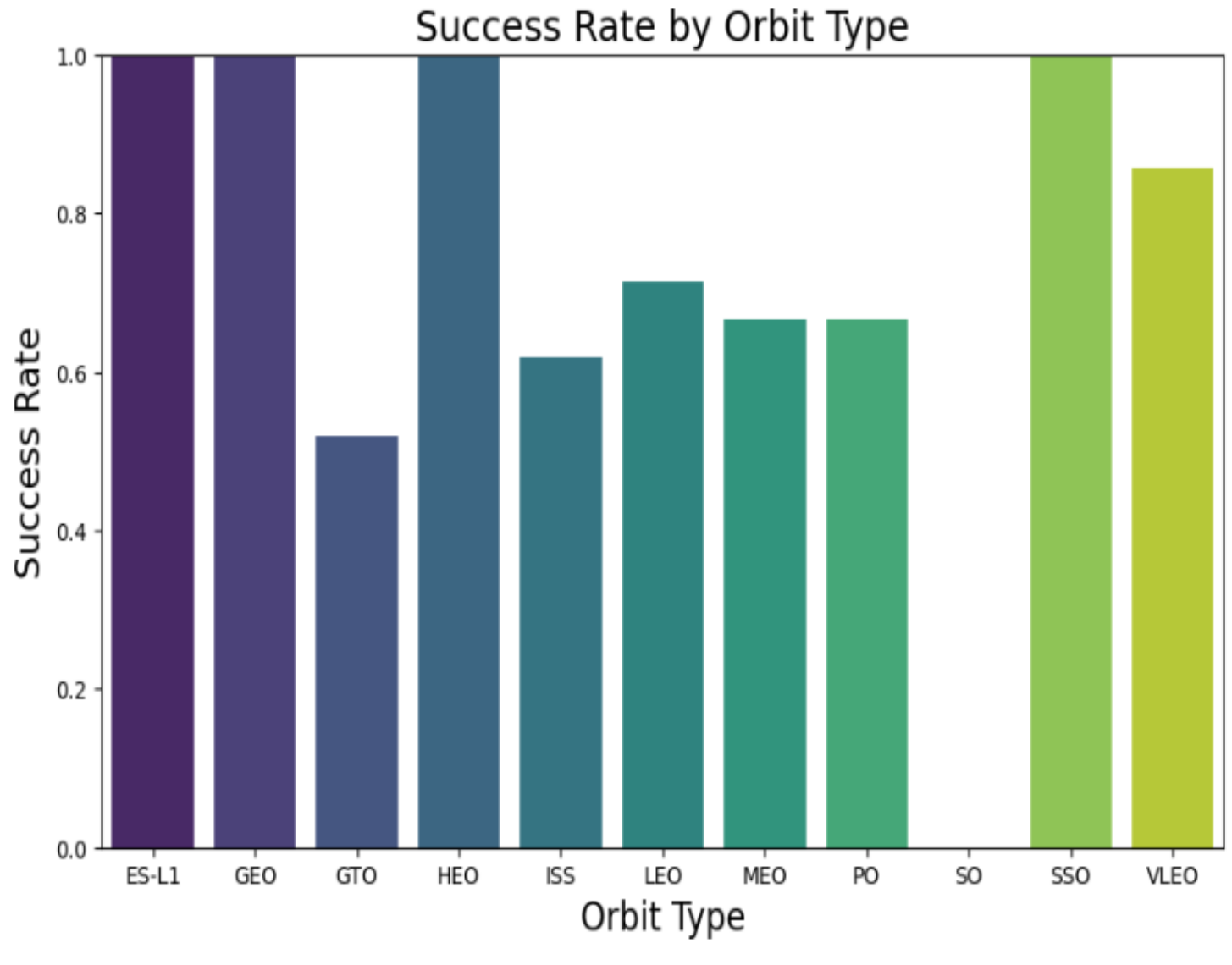
Orbit Type :

Success Rate vs Orbit Type (Bar Chart)

- **High Success Rates**: Certain orbit types such as LEO and ISS show higher success rates, indicating that launches to these orbits are more consistent and reliable.
- **Low Success Rates:** Orbit types like GTO tend to show lower success rates, suggesting these missions are more complex and may involve greater technical challenges.

Activity :

- Created a bar chart to compare the launch success rate for each orbit type. The x-axis represents the orbit types, while the y-axis represents the success rate.
- This visualization helps clearly identify which orbit missions have higher or lower landing success probabilities.



Flight Number vs. Orbit Type

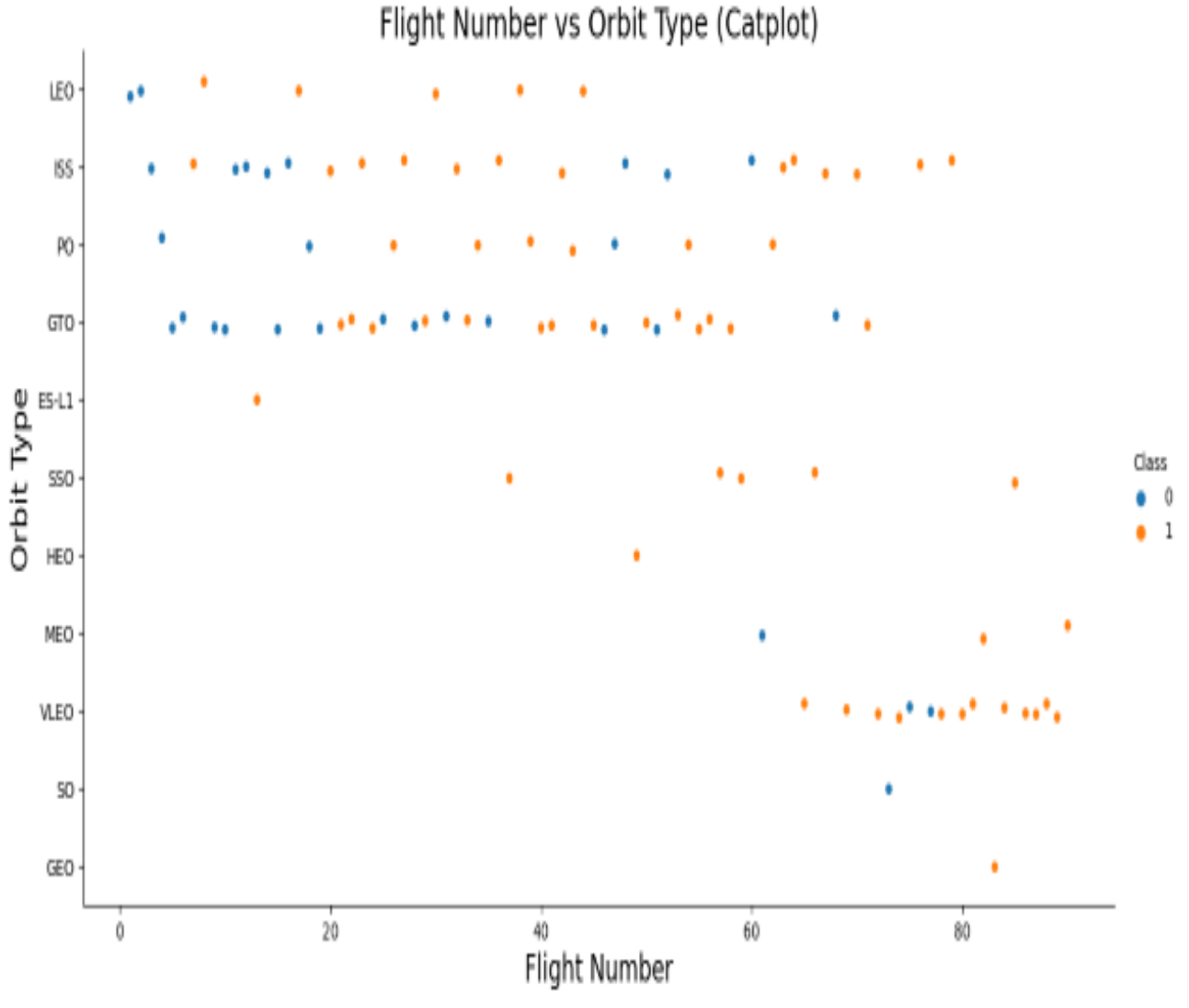
Flight Number vs Orbit Type (Scatter Plot)

Activity:

- Created a scatter plot to analyze the relationship between Flight Number and Orbit Type.
- The x-axis represents the Flight Number (chronological order of launches). The y-axis represents the Orbit Type.
- Each point represents an individual SpaceX launch mission.

Outcome:

- The plot shows how different orbit missions are distributed across the sequence of launches.
- Earlier flights appear concentrated in fewer orbit types, while later flights show more diverse orbit missions.
- This indicates that SpaceX expanded its mission capabilities over time, launching payloads to a wider range of orbits.



Payload vs. Orbit Type

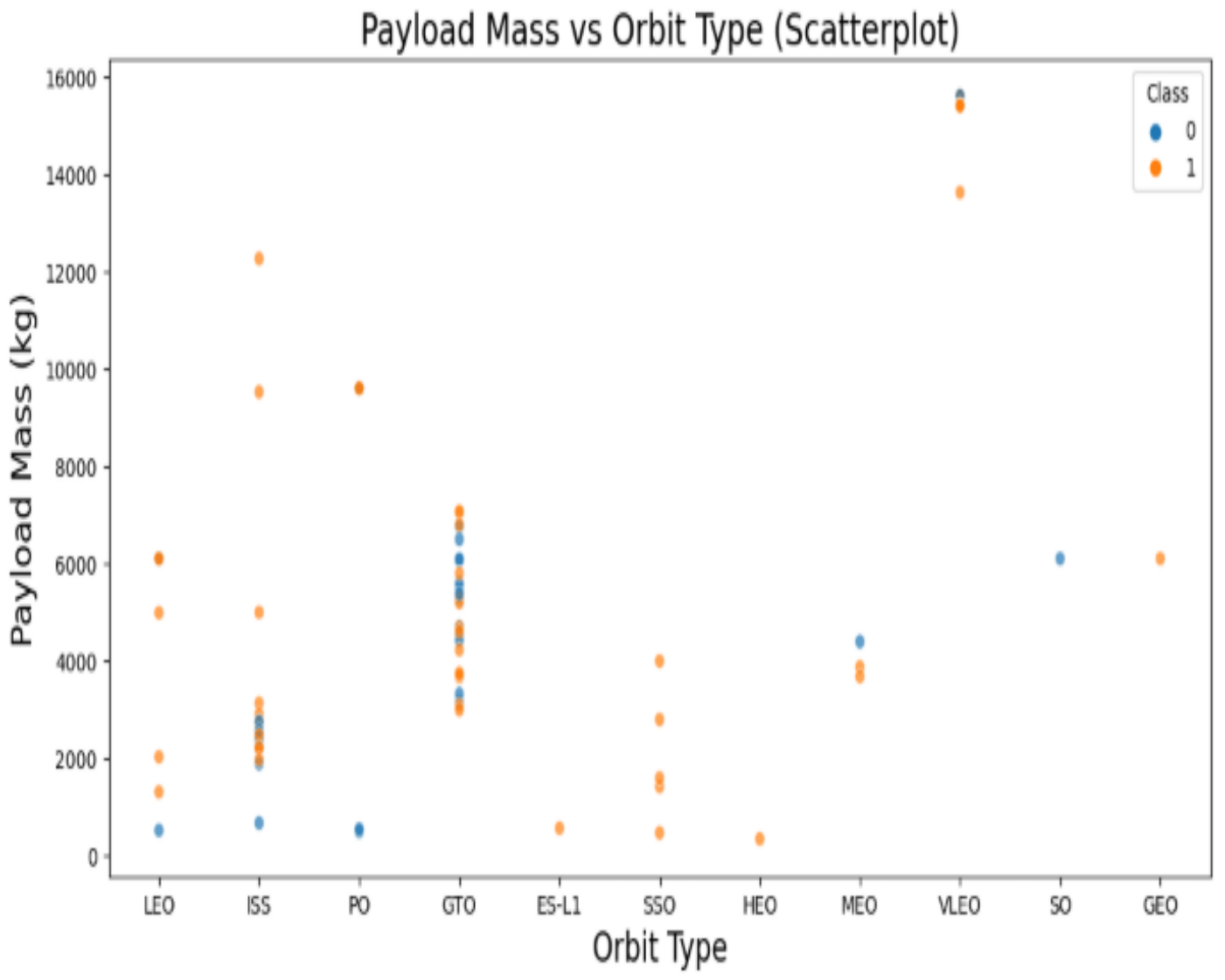
Payload vs Orbit Type (Scatter Plot)

Activity:

- Created a scatter plot to examine the relationship between Payload Mass and Orbit Type. The x-axis represents the Orbit Type, and the y-axis represents the Payload Mass (kg).
- Each point corresponds to an individual SpaceX launch.

Outcome:

- The plot shows how payload sizes vary across different orbit missions.
- Some orbit types contain heavier payload launches, indicating missions that require larger payload capacities.
- Other orbit types show smaller and more consistent payload sizes, suggesting more standardized mission profiles.



Launch Success Yearly Trend

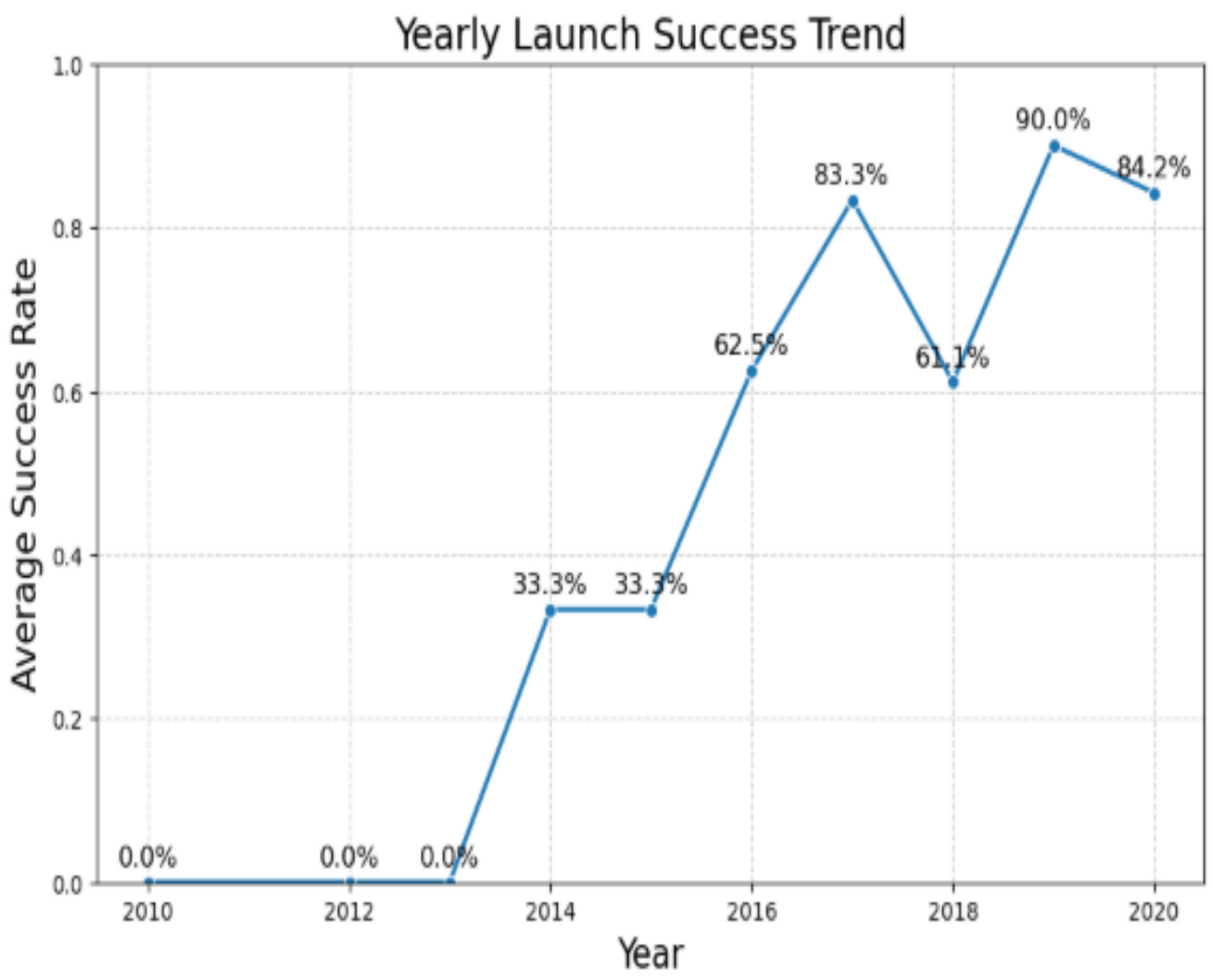
Yearly Average Success Rate (Line Chart)

Activity:

- Created a line chart to analyze the average launch success rate per year. The x-axis represents the Year, and the y-axis represents the average success rate.
- Each point on the line shows the average landing success rate for launches in that year.

Outcome:

- The chart shows how the success rate has changed over time.
- Early years show lower or inconsistent success rates, while later years show higher and more stable success rates.
- This trend suggests that SpaceX improved its launch and landing technology over the years, leading to better mission reliability.



you can observe that the sucess rate since 2013 kept increasing till 2020

All Launch Site Names

Display the names of the unique launch sites in the space mission

In [11]:
```
print(df.columns)
```

```
Index(['Date', 'Time (UTC)', 'Booster_Version', 'Launch_Site', 'Payload',
       'PAYLOAD_MASS__KG_', 'Orbit', 'Customer', 'Mission_Outcome',
       'Landing_Outcome'],
      dtype='object')
```

In [12]:
```
# Unique launch sites
unique_sites = df['Launch_Site'].unique()
print(unique_sites)
```

```
['CCAFS LC-40' 'VAFB SLC-4E' 'KSC LC-39A' 'CCAFS SLC-40']
```

In [13]:
```
# Run your query
%sql SELECT DISTINCT Launch_Site FROM SPACEXTBL;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[13]:

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

Display 5 records where launch sites begin with the string 'CCA'

In [14]:

```
# Filter rows where Launch_Site begins with 'CCA'
cca_records = df[df['Launch_Site'].str.startswith('CCA')]

# Display first 5 records
print(cca_records.head(5))
```

```
        Date Time (UTC) Booster_Version  Launch_Site  \
0  2010-06-04   18:45:00  F9 v1.0  B0003  CCAFS LC-40
1  2010-12-08   15:43:00  F9 v1.0  B0004  CCAFS LC-40
2  2012-05-22    7:44:00  F9 v1.0  B0005  CCAFS LC-40
3  2012-10-08    0:35:00  F9 v1.0  B0006  CCAFS LC-40
4  2013-03-01   15:10:00  F9 v1.0  B0007  CCAFS LC-40

                                      Payload  PAYLOAD_MASS__KG_  \
0              Dragon Spacecraft Qualification Unit                  0
1  Dragon demo flight C1, two CubeSats, barrel of...                 0
2                           Dragon demo flight C2                525
3                                     SpaceX CRS-1                500
4                                     SpaceX CRS-2                677

       Orbit         Customer Mission_Outcome      Landing_Outcome
0        LEO           SpaceX         Success  Failure (parachute)
1  LEO (ISS)  NASA (COTS) NRO         Success  Failure (parachute)
2  LEO (ISS)      NASA (COTS)         Success           No attempt
3  LEO (ISS)       NASA (CRS)         Success           No attempt
4  LEO (ISS)       NASA (CRS)         Success           No attempt
```

In [15]:

```
%%sql SELECT * FROM SPACEXTBL
WHERE Launch_Site LIKE 'CCA%'
LIMIT 5;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[15]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

Display the total payload mass carried by boosters launched by NASA (CRS)

In [16]:

```
# Filter for NASA (CRS) launches
nasa_crs = df[df['Customer'] == 'NASA (CRS)']

# Calculate total payload mass
total_payload = nasa_crs['PAYLOAD_MASS__KG_'].sum()
print("Total payload mass carried by NASA (CRS) boosters:", total_payload, "kg")
```

```
Total payload mass carried by NASA (CRS) boosters: 45596 kg
```

In [17]:

```
%%sql SELECT SUM(PAYLOAD_MASS__KG_) AS TotalPayloadMass
FROM SPACEXTBL
WHERE Customer = 'NASA (CRS)';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[17]:

TotalPayloadMass
45596

Average Payload Mass by F9 v1.1

Display average payload mass carried by booster version F9 v1.1

In [18]:

```
# Filter for booster version F9 v1.1
f9_v11 = df[df['Booster_Version'] == 'F9 v1.1']

# Calculate average payload mass
avg_payload = f9_v11['PAYLOAD_MASS__KG_'].mean()
print("Average payload mass carried by F9 v1.1 boosters:", avg_payload, "kg")
```

```
Average payload mass carried by F9 v1.1 boosters: 2928.4 kg
```

In [19]:

```
%%sql SELECT AVG(PAYLOAD_MASS__KG_) AS AvgPayloadMass
FROM SPACEXTBL
WHERE Booster_Version = 'F9 v1.1';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[19]:

AvgPayloadMass
2928.4

First Successful Ground Landing Date

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

[20]:

```
# Filter for successful ground pad landings
ground_pad_success = df[df['Landing_Outcome'] == 'Success (ground pad)']

# Find the earliest date
first_success_date = ground_pad_success['Date'].min()
print("First successful ground pad landing date:", first_success_date)
```

```
First successful ground pad landing date: 2015-12-22
```

[21]:

```
%%sql SELECT MIN(Date) AS FirstSuccessDate
FROM SPACEXTBL
WHERE Landing_Outcome = 'Success (ground pad)';
```

```
 * sqlite:///my_data1.db
Done.
```

[21]:

FirstSuccessDate
2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

In [22]:

```
# Filter boosters with success in drone ship and payload mass between 4000 and 6000
filtered_boosters = df[
    (df['Landing_Outcome'] == 'Success (drone ship)') &
    (df['PAYLOAD_MASS__KG_'] > 4000) &
    (df['PAYLOAD_MASS__KG_'] < 6000)
]

# Display the booster names
print(filtered_boosters['Booster_Version'].unique())
```

```
['F9 FT B1022' 'F9 FT B1026' 'F9 FT  B1021.2' 'F9 FT  B1031.2']
```

In [23]:

```
%%sql SELECT DISTINCT Booster_Version
FROM SPACEXTBL
WHERE Landing_Outcome = 'Success (drone ship)'
  AND PAYLOAD_MASS__KG_ > 4000
  AND PAYLOAD_MASS__KG_ < 6000;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[23]:

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

List the total number of successful and failure mission outcomes

In [24]:

```
# Count mission outcomes
outcome_counts = df['Mission_Outcome'].value_counts()
print(outcome_counts)
```

```
Success                             98
Failure (in flight)                  1
Success (payload status unclear)     1
Success                              1
Name: Mission_Outcome, dtype: int64
```

In [25]:

```
%%sql SELECT Mission_Outcome, COUNT(*) AS OutcomeCount
FROM SPACEXTBL
GROUP BY Mission_Outcome
ORDER BY OutcomeCount DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[25]:

Mission_Outcome	OutcomeCount
Success	98
Success (payload status unclear)	1
Success	1
Failure (in flight)	1

Boosters Carried Maximum Payload

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

In [26]:

```
# Find the maximum payload mass
max_payload = df['PAYLOAD_MASS__KG_'].max()

# Get booster versions that carried this payload
boosters = df[df['PAYLOAD_MASS__KG_'] == max_payload]['Booster_Version'].unique()
print("Boosters with maximum payload mass:", boosters)
```

```
Boosters with maximum payload mass: ['F9 B5 B1048.4' 'F9 B5 B1049.4' 'F9 B5 B1051.3' 'F9 B5 B1056.4'
 'F9 B5 B1048.5' 'F9 B5 B1051.4' 'F9 B5 B1049.5' 'F9 B5 B1060.2 '
 'F9 B5 B1058.3 ' 'F9 B5 B1051.6' 'F9 B5 B1060.3' 'F9 B5 B1049.7 ']
```

In [27]:

```
%%sql SELECT DISTINCT Booster_Version
FROM SPACEXTBL
WHERE PAYLOAD_MASS__KG_ = (
    SELECT MAX(PAYLOAD_MASS__KG_)
    FROM SPACEXTBL
);
```

```
 * sqlite:///my_data1.db
Done.
```

Out[27]:

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

In [28]:

```
import pandas as pd

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Filter for year 2015 and failure in drone ship
filtered = df[
    (df['Date'].dt.year == 2015) &
    (df['Landing_Outcome'].str.contains('Failure (drone ship)', na=False, regex=False))
]

# Add month names
filtered['MonthName'] = filtered['Date'].dt.month_name()

# Select relevant columns
result = filtered[['MonthName', 'Booster_Version', 'Launch_Site', 'Landing_Outcome']]

print(result)
```

```
   MonthName Booster_Version  Launch_Site       Landing_Outcome
13   January   F9 v1.1 B1012  CCAFS LC-40  Failure (drone ship)
16     April   F9 v1.1 B1015  CCAFS LC-40  Failure (drone ship)
```

In [29]:

```
%%sql SELECT
    CASE substr(Date, 6, 2)
        WHEN '01' THEN 'January'
        WHEN '02' THEN 'February'
        WHEN '03' THEN 'March'
        WHEN '04' THEN 'April'
        WHEN '05' THEN 'May'
        WHEN '06' THEN 'June'
        WHEN '07' THEN 'July'
        WHEN '08' THEN 'August'
        WHEN '09' THEN 'September'
        WHEN '10' THEN 'October'
        WHEN '11' THEN 'November'
        WHEN '12' THEN 'December'
    END AS MonthName,
    Booster_Version,
    Launch_Site,
    Landing_Outcome
FROM SPACEXTBL
WHERE substr(Date, 1, 4) = '2015'
  AND Landing_Outcome LIKE '%Failure%'
  AND Landing_Outcome LIKE '%drone ship%';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[29]:

MonthName	Booster_Version	Launch_Site	Landing_Outcome
January	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
April	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```python
import pandas as pd

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Filter rows between the given date range
mask = (df['Date'] >= '2010-06-04') & (df['Date'] <= '2017-03-20')
filtered = df[mask]

# Count landing outcomes and sort in descending order
outcome_counts = filtered['Landing_Outcome'].value_counts().sort_values(ascending=False)

print(outcome_counts)
```

```
No attempt                10
Failure (drone ship)       5
Success (drone ship)       5
Controlled (ocean)         3
Success (ground pad)       3
Failure (parachute)        2
Uncontrolled (ocean)       2
Precluded (drone ship)     1
Name: Landing_Outcome, dtype: int64
```

```cpp
%%sql SELECT Landing_Outcome, COUNT(*) AS OutcomeCount
FROM SPACEXTBL
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome
ORDER BY OutcomeCount DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

Landing_Outcome	OutcomeCount
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Section 3

Launch Sites Proximities Analysis

Global Map of SpaceX Launch Sites

Explanation of the Map

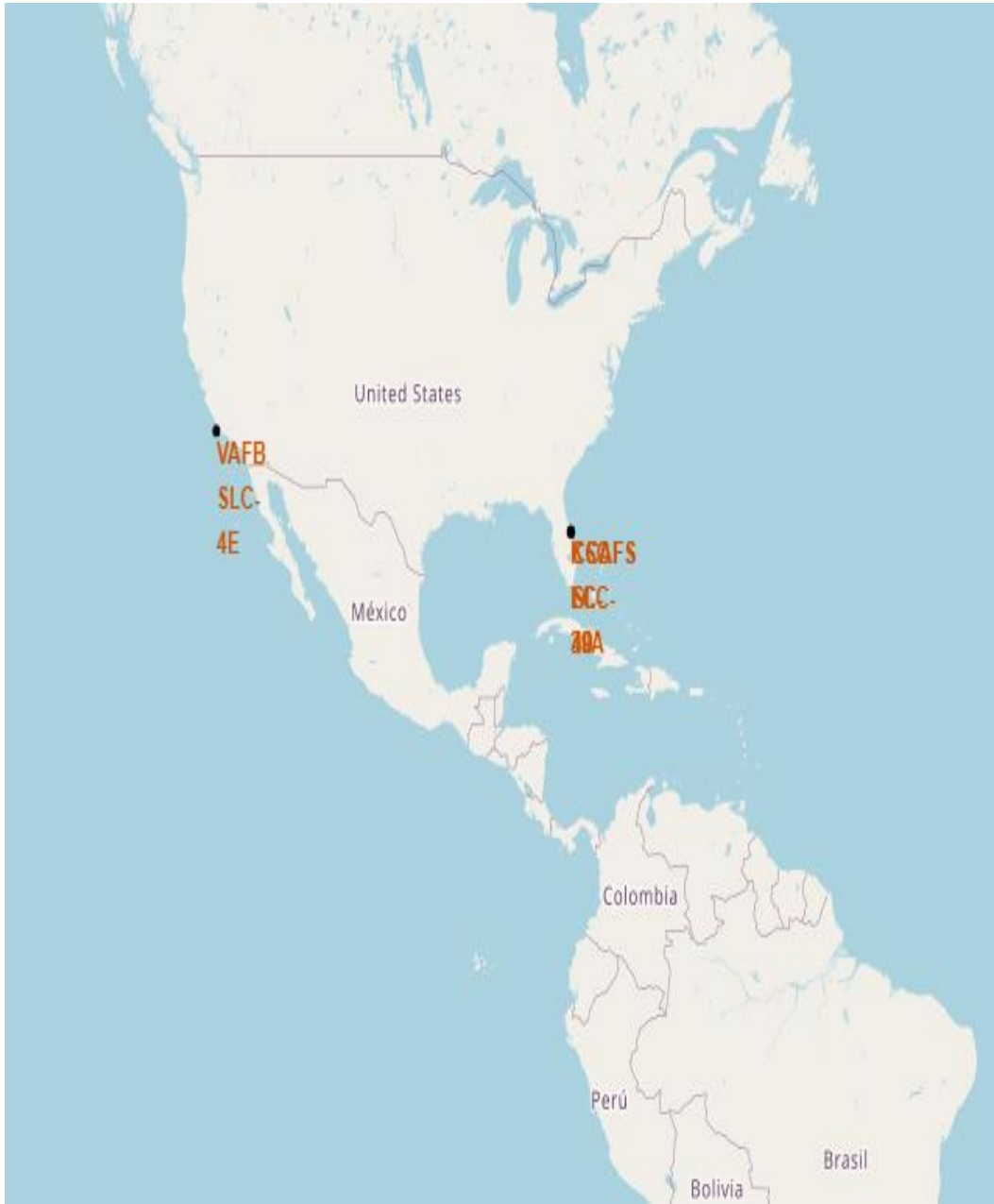
- The Folium map displays the geographical locations of SpaceX launch sites around the world using location markers.Each marker represents a launch site, allowing users to visually identify where launches take place. The map uses latitude and longitude coordinates to accurately position each launch site on the global map.

Important Elements :

- Location Markers: Show the exact positions of the launch sites such as CCAFS LC-40, KSC LC-39A, and VAFB SLC-4E.
- Interactive Map: Users can zoom in and out to explore the launch site locations more clearly.
- Pop-up Labels: Clicking on a marker displays the name of the launch site.

Key Findings :

- Most launch sites are located in the United States, mainly in Florida and California.
- Florida hosts multiple launch sites, indicating it is a major hub for SpaceX launches.
- The map helps visualize the geographical distribution of launch operations, making it easier to understand where SpaceX conducts its missions.



Launch Outcomes Visualization on SpaceX Launch Site Map

Explanation of the Map :

The Folium map displays SpaceX launch sites along with their launch outcomes, using different colors to represent success or failure. Each launch is represented by a color-coded marker placed at the location of the launch site.

Important Elements :

1] Color-Coded Markers: Different colors represent successful and unsuccessful launches, making it easier to distinguish outcomes.

2] Launch Site Locations: Markers are placed on the exact coordinates of each launch site.

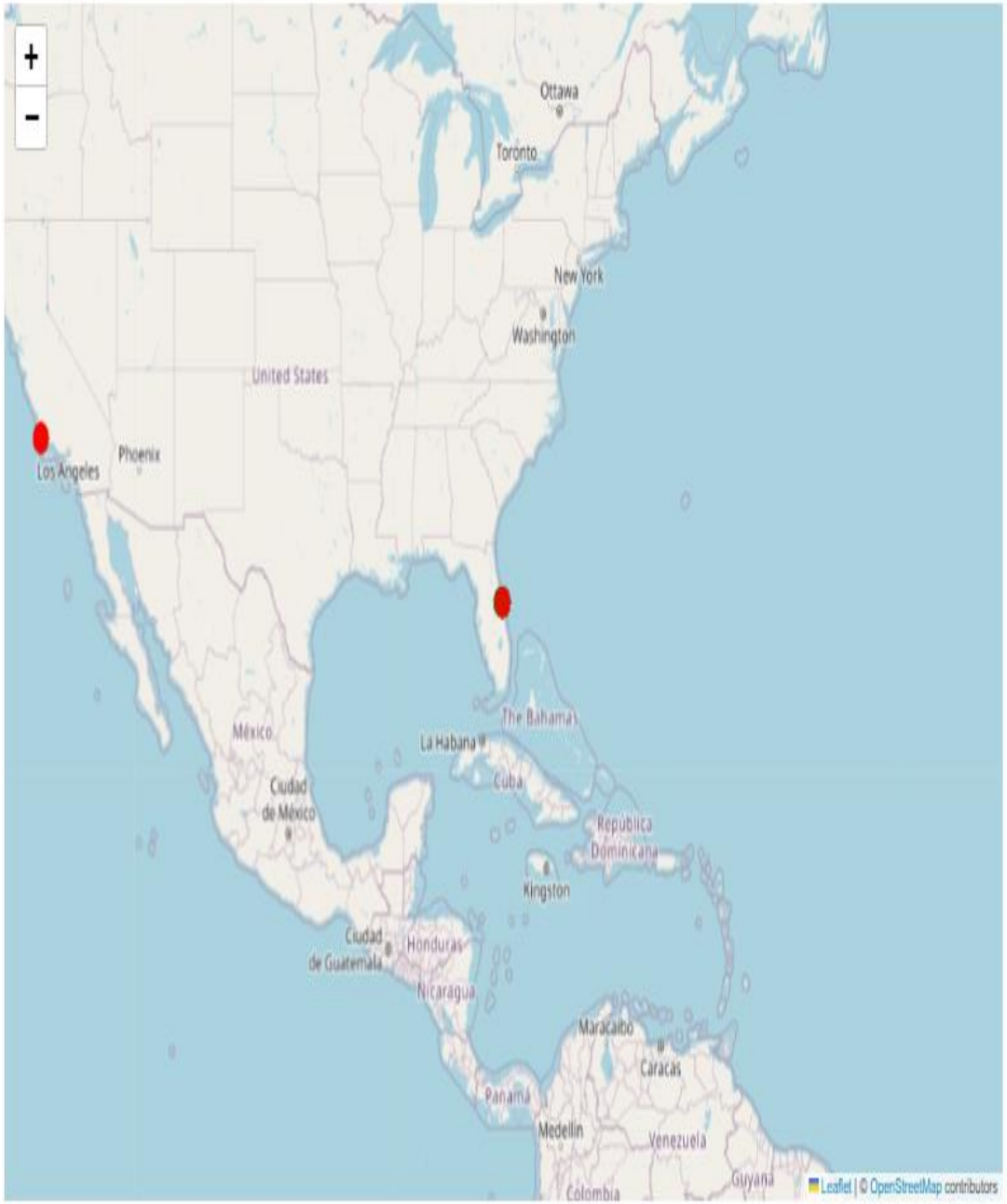
3] Interactive Features: The map allows zooming and clicking on markers to view additional information about each launch.

Key Findings :

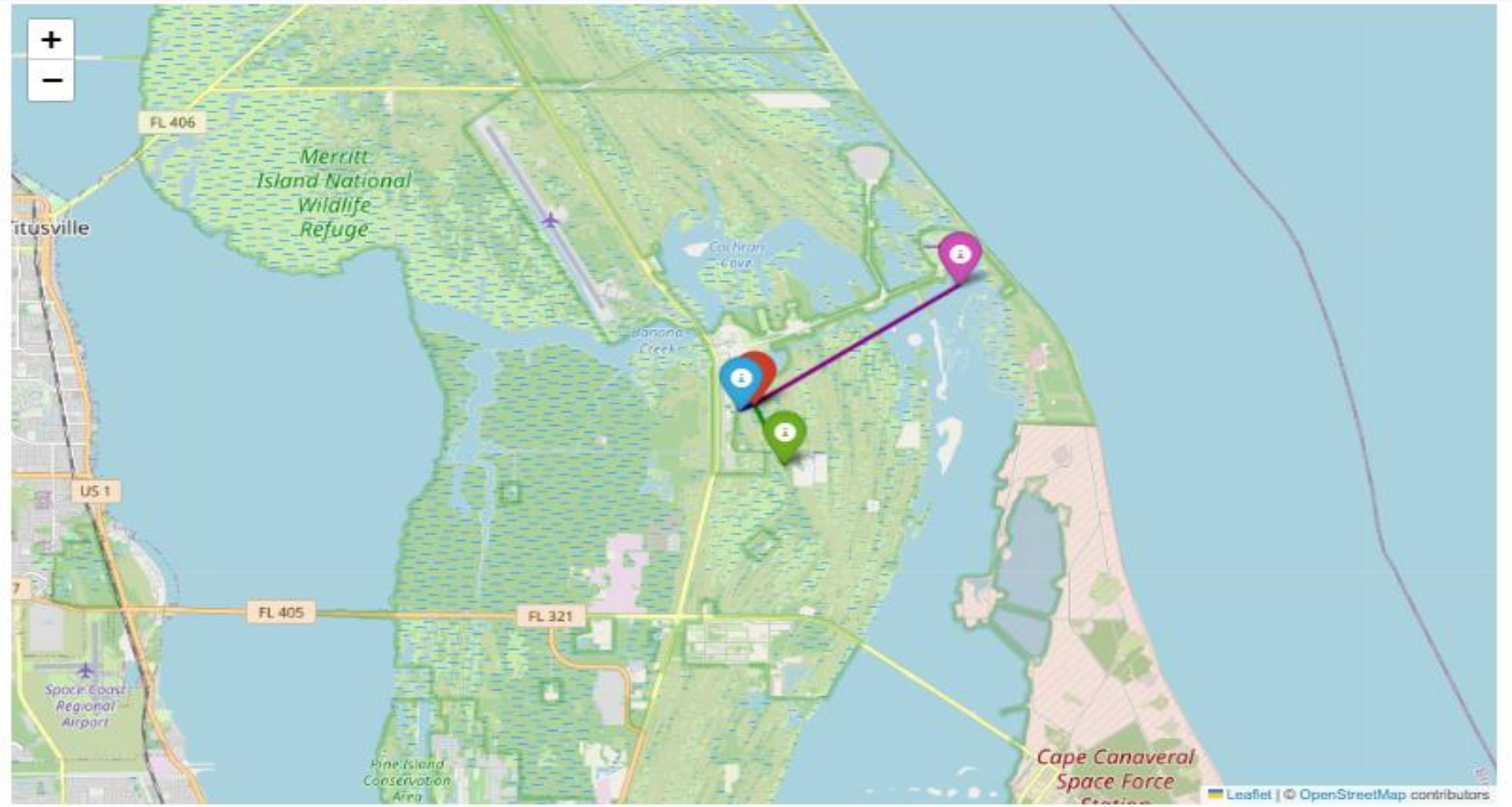
- Successful launches appear more frequently, indicating a generally high success rate for SpaceX missions.

- Launch sites such as KSC LC-39A and CCAFS SLC-40 show many successful outcomes, highlighting their operational reliability.

- The visualization helps quickly identify patterns of launch performance across different locations.



Proximity Analysis of SpaceX Launch Site and Nearby Infrastructure



Proximity Analysis of SpaceX Launch Site and Nearby Infrastructure

Explanation of the Map :

The Folium map highlights a selected SpaceX launch site along with nearby geographical features such as railways, highways, and coastlines. Lines are drawn from the launch site to nearby infrastructure, and the distance between them is calculated and displayed on the map. This helps visualize how close important transportation routes and natural features are to the launch site.

Important Elements :

1] Launch Site Marker: Indicates the exact location of the selected SpaceX launch facility.
2] Nearby Infrastructure Markers: Show nearby railway lines, highways, and coastline locations.
3] Distance Lines: Lines drawn between the launch site and nearby locations show the distance measurements.
4] Interactive Map Features: Users can zoom in, click markers, and view distance information for each nearby feature.

Key Findings :

The launch site is located close to the coastline, which is important for safe rocket launches and debris management.
Highways and railways are nearby, providing convenient transportation and logistics support for moving equipment and rocket components.
The map demonstrates how launch sites are strategically located near transportation infrastructure while maintaining safe distances from populated areas.

Section 4

Build a Dashboard with Plotly Dash

Launch Success Distribution for All Launch Sites

Important Elements in the Screenshot :

1. Dashboard Title :

The dashboard title “SpaceX Launch Records Dashboard” appears at the top and indicates that the visualizations analyze SpaceX launch data.

2. Launch Site Dropdown:

A dropdown menu allows users to select a specific launch site or view all sites together. When All Sites is selected, the pie chart shows the total successful launches across all locations.

3. Pie Chart Visualization :

The pie chart represents the proportion of successful launches for each launch site.

Each slice corresponds to a launch site:

CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40
The size of each slice indicates the relative number of successful launches.

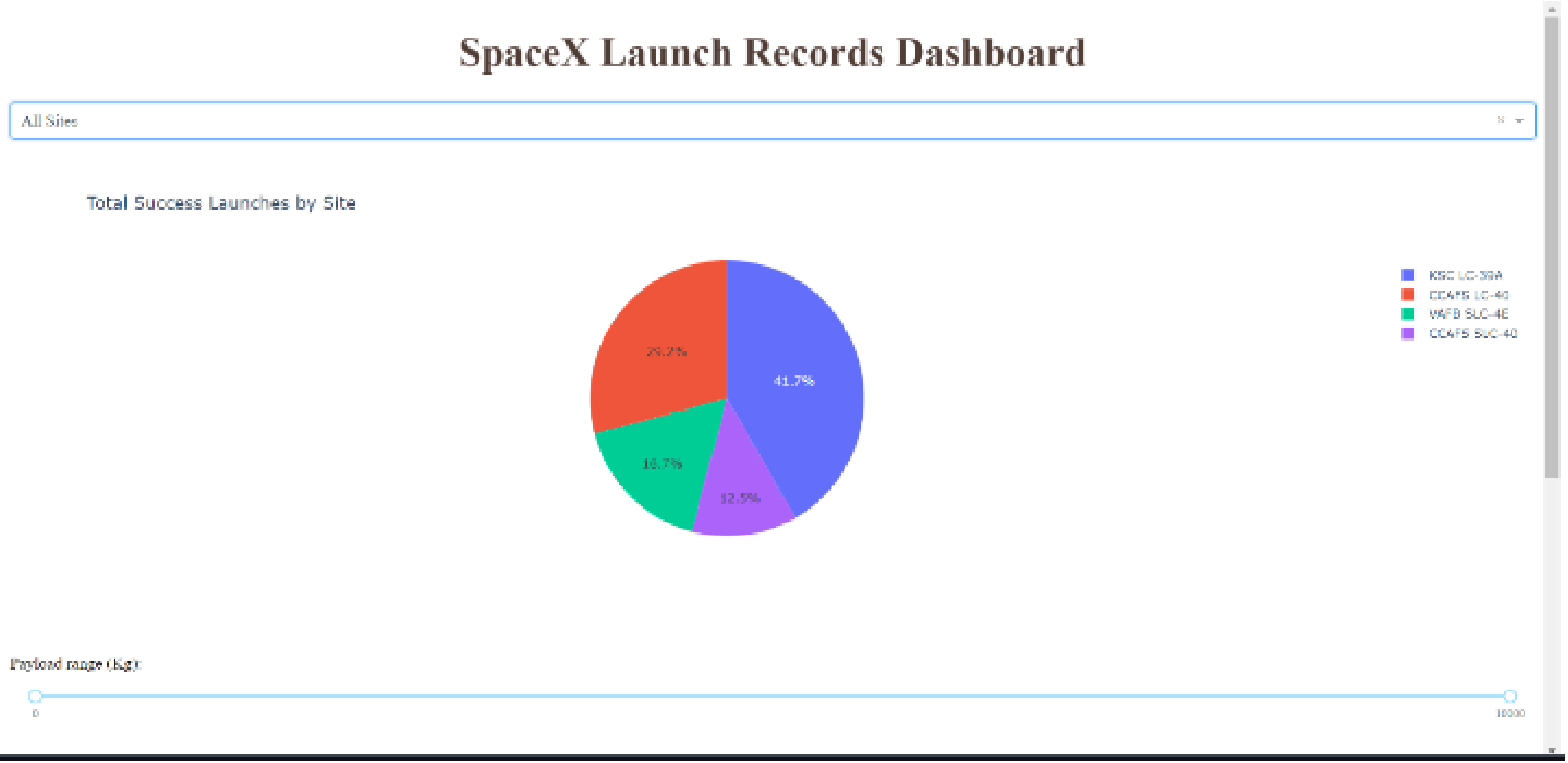
4. Legend

The legend identifies each launch site with a distinct color, making it easy to compare the launch success distribution across sites.

Key Findings :

- Some launch sites contribute more successful launches than others, which can be observed from the larger slices of the pie chart.
- Launch sites like KSC LC-39A and CCAFS LC-40 show a higher proportion of successful launches, indicating they are frequently used and have strong success records.
- The visualization helps quickly identify which launch sites have been most successful in SpaceX missions.

Launch Success Distribution for All Launch Sites



Launch Success vs Failure for the Site with the Highest Success Ratio (KSC LC-39A)

Screenshot Description

This screenshot shows a pie chart displaying the distribution of successful and failed launches for the launch site with the highest success ratio. The visualization is generated using Plotly in a dashboard built with Dash.

The launch site selected in the dropdown is Kennedy Space Center Launch Complex 39A (KSC LC-39A), which has the highest proportion of successful launches in the dataset used by SpaceX.

Important Elements in the Screenshot :

1. Launch Site Dropdown :

The dropdown menu is set to KSC LC-39A, allowing the dashboard to filter data and display launch statistics only for that specific site.

2. Pie Chart Visualization :

The pie chart shows two categories:
Successful launches (class = 1)
Failed launches (class = 0)
Each slice represents the proportion of successful and unsuccessful launches at this launch site.

3. Color-coded Segments :

The chart uses different colors to distinguish between success and failure outcomes, making it easy to compare the results visually.

4. Legend :

The legend indicates which color corresponds to successful launches and which corresponds to failed launches, helping users interpret the chart quickly.

Launch Success vs Failure for the Site with the Highest Success Ratio (KSC LC-39A)

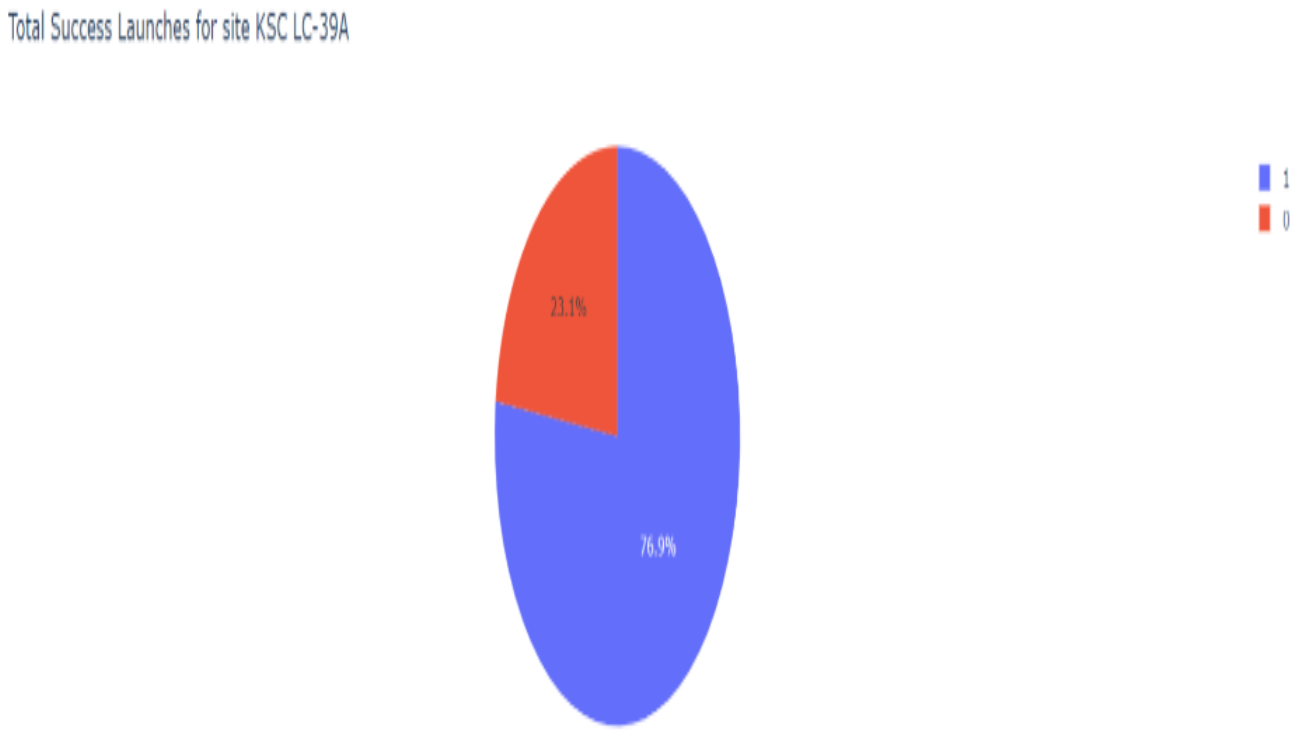
Key Findings :

- KSC LC-39A has the highest launch success ratio among all launch sites in the dataset.

- The pie chart shows that the majority of launches from this site were successful, indicating strong performance and reliability.

- The relatively small failure segment suggests that launches from this site are more likely to succeed compared to other launch locations.

- This analysis highlights the importance of launch site selectior in mission success for SpaceX.



Payload Mass vs Launch Outcome Across All Launch Sites

Screenshot Description :

This screenshot shows a scatter plot illustrating the relationship between payload mass and launch success for all launch sites. The visualization was created using Plotly within an interactive dashboard built using Dash.
The scatter plot helps analyze how payload weight affects the probability of launch success and how different booster versions perform across payload ranges. The data represents launch records from SpaceX.

Important Elements in the Screenshot :

1. Payload Range Slider :

The payload range slider allows users to filter launches by payload mass (kg). By adjusting the slider, users can observe how launch success varies for different payload ranges.

2. Scatter Plot Visualization :

Each point in the scatter plot represents a single launch.
X-axis: Payload Mass (kg)
Y-axis: Launch Outcome (0 = failure, 1 = success)
This visualization helps identify trends between payload weight and mission success.

3. Color-coded Booster Versions :

The points are colored according to the Booster Version Category, which helps distinguish between different rocket versions used during launches.
Examples include:
v1.0, v1.1, FT (Full Thrust), B4, B5

Payload Mass vs Launch Outcome Across All Launch Sites

4. Launch Site Coverage :

Since All Sites is selected in the dropdown, the scatter plot includes launches from multiple SpaceX launch sites, enabling a broader analysis of launch performance.

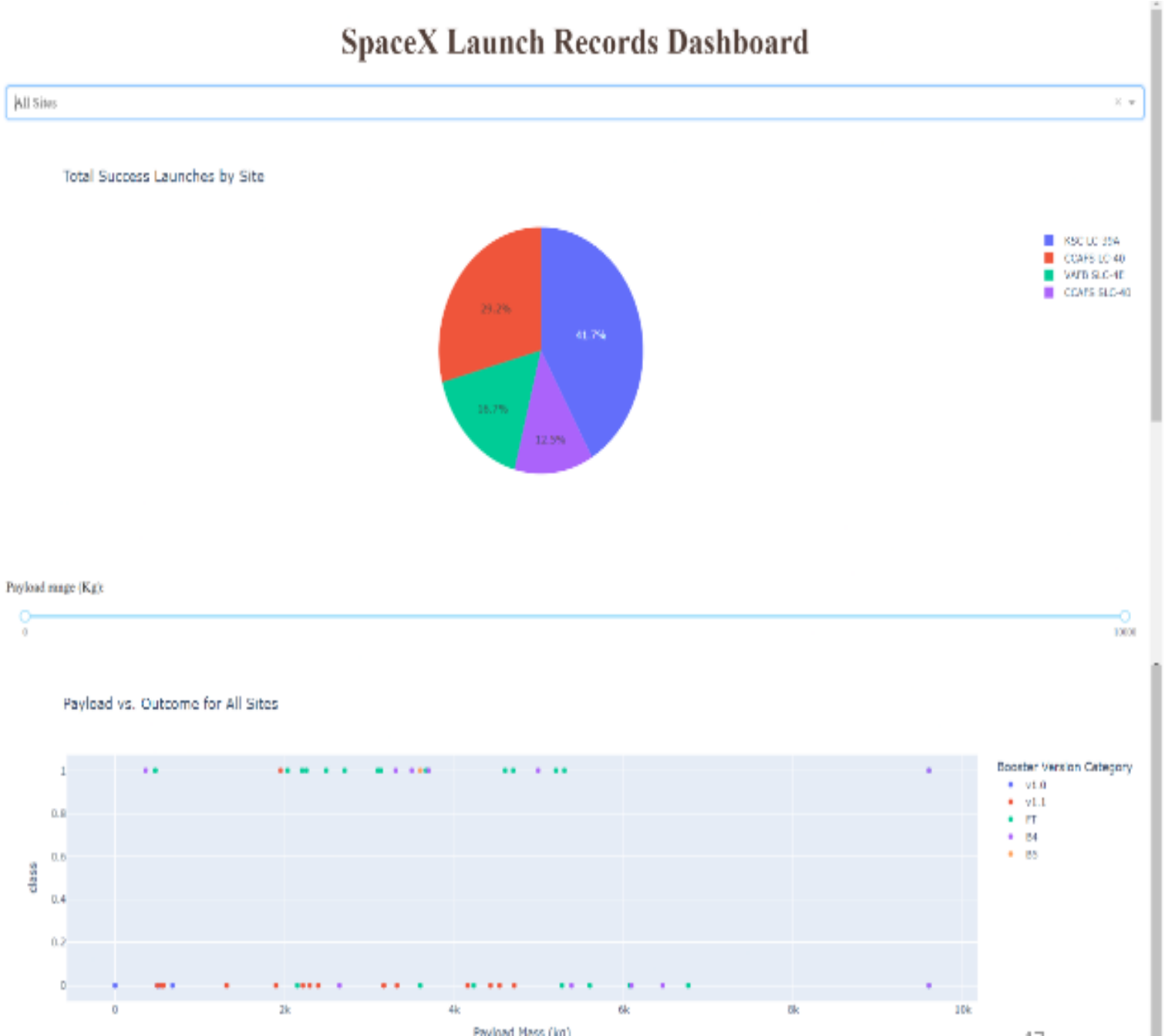
Key Findings :

- Launches carrying medium to high payload masses (approximately 3000–6000 kg) tend to show higher success rates.

- Newer booster versions such as B4 and B5 appear more frequently among successful launches, indicating improvements in rocket technology over time.

- Lower payload ranges include both successful and failed launches, suggesting that payload mass alone does not fully determine launch success.

- The scatter plot demonstrates that advancements in booster versions contribute significantly to improved launch success rates for SpaceX missions.

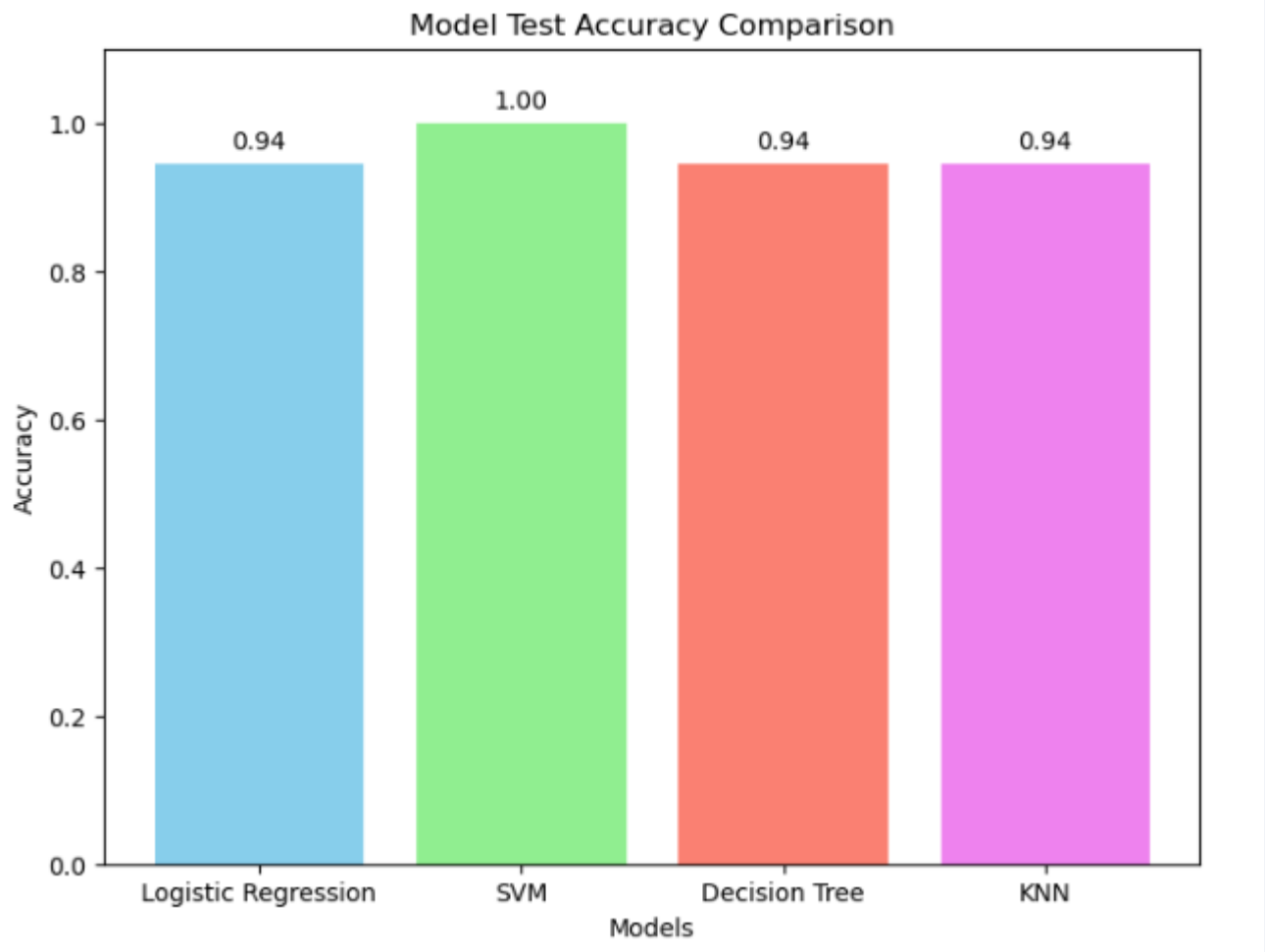


Section 5

Predictive Analysis (Classification)

Classification Accuracy

- The classification models built include Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN). After evaluating the models, the SVM classifier achieved the highest accuracy of 1.00, while the other models achieved accuracies of 0.94. Therefore, the SVM model performed best for predicting launch success in the dataset from SpaceX.



Confusion Matrix

Explanation of the Results :

The confusion matrix shows that the Support Vector Machine (SVM) model perfectly classified all test samples.

There were no misclassifications, meaning the model correctly predicted both successful and failed launches.

This explains why the model achieved 100% accuracy when predicting launch outcomes for SpaceX missions.

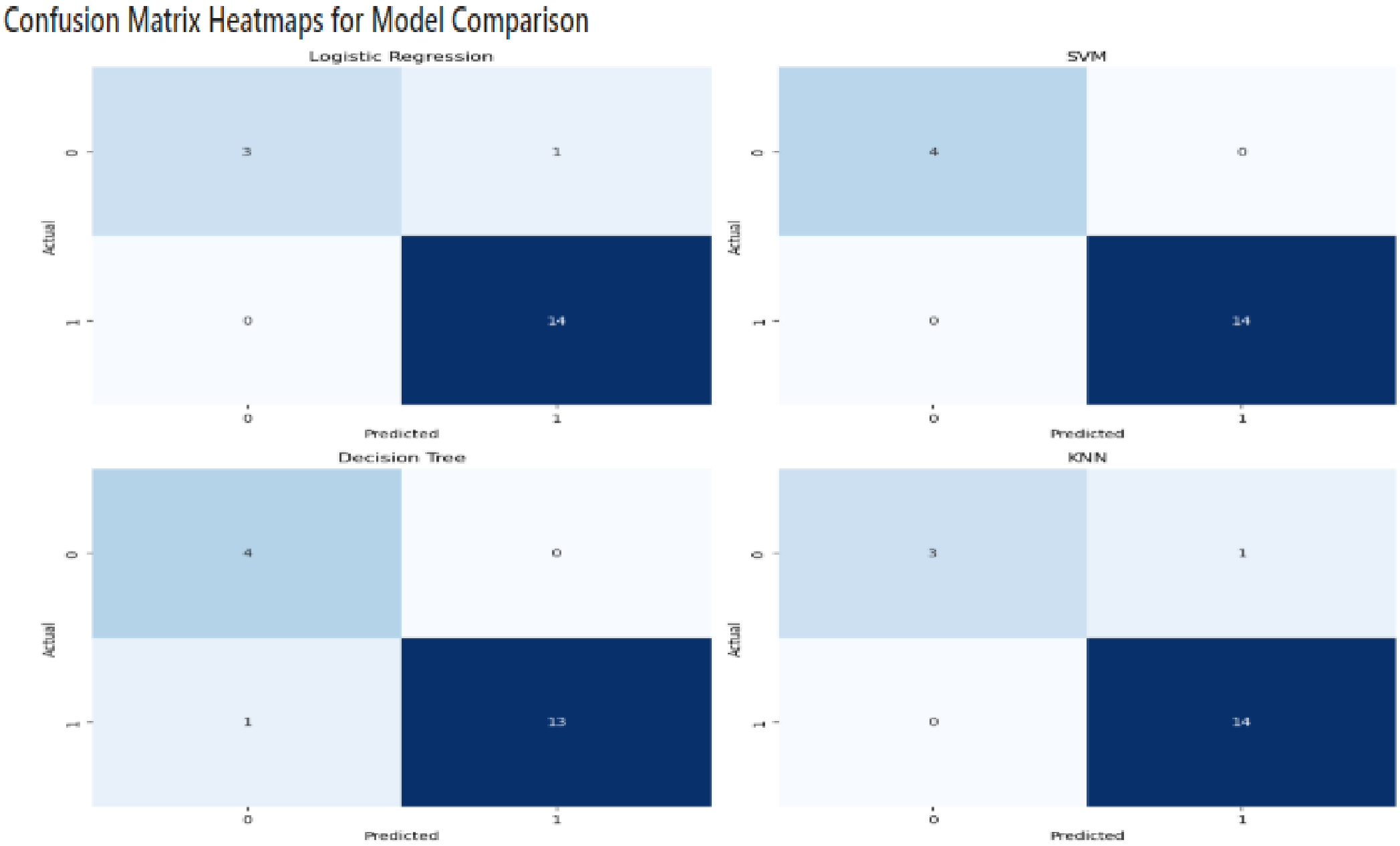
The results suggest that the SVM model captured the relationships between the features and launch success very effectively for this dataset.

Find the method performs best:

```
]:
# Compare test accuracies of all models
logreg_acc = logreg_cv.score(X_test, Y_test)
svm_acc = svm_cv.score(X_test, Y_test)
tree_acc = tree_cv.score(X_test, Y_test)
knn_acc = knn_cv.score(X_test, Y_test)

# Print results
print("Logistic Regression Test Accuracy:", logreg_acc)
print("SVM Test Accuracy:", svm_acc)
print("Decision Tree Test Accuracy:", tree_acc)
print("KNN Test Accuracy:", knn_acc)

# Find the best performing model
accuracies = {
    "Logistic Regression": logreg_acc,
    "SVM": svm_acc,
    "Decision Tree": tree_acc,
    "KNN": knn_acc
}

best_model = max(accuracies, key=accuracies.get)
print("Best performing model:", best_model, "with accuracy:", accuracies[best_model])
```

```
Logistic Regression Test Accuracy: 0.9444444444444444
SVM Test Accuracy: 1.0
Decision Tree Test Accuracy: 0.9444444444444444
KNN Test Accuracy: 0.9444444444444444
Best performing model: SVM with accuracy: 1.0
```

Confusion Matrix



Conclusions

- In this project, several classification models were developed to predict the success of rocket launches using historical launch data from SpaceX. The models evaluated included Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN).

- After training and testing the models, their performance was compared using classification accuracy. Among all the models, the Support Vector Machine (SVM) model achieved the highest accuracy of 1.00, while Logistic Regression, Decision Tree, and KNN achieved accuracies of 0.94. This indicates that the SVM model was the most effective for predicting launch success in this dataset.

- The confusion matrix of the best-performing model further confirmed that the SVM classifier correctly predicted all the test samples without misclassification, resulting in perfect accuracy.

- Overall, the results demonstrate that machine learning models can effectively analyze launch parameters such as payload mass, launch site, and booster version to predict launch outcomes. The analysis highlights the usefulness of data-driven approaches in improving decision-making and performance evaluation in aerospace missions.

- This project illustrates how classification algorithms can be applied to real-world datasets to gain insights and

Conclusions

- In this project, several classification models were developed to predict the success of rocket launches using historical launch data from SpaceX. The models evaluated included Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN).

- After training and testing the models, their performance was compared using classification accuracy. Among all the models, the Support Vector Machine (SVM) model achieved the highest accuracy of 1.00, while Logistic Regression, Decision Tree, and KNN achieved accuracies of 0.94. This indicates that the SVM model was the most effective for predicting launch success in this dataset.

- The confusion matrix of the best-performing model further confirmed that the SVM classifier correctly predicted all the test samples without misclassification, resulting in perfect accuracy.

- Overall, the results demonstrate that machine learning models can effectively analyze launch parameters such as payload mass, launch site, and booster version to predict launch outcomes. The analysis highlights the usefulness of data-driven approaches in improving decision-making and performance evaluation in aerospace missions.

- This project illustrates how classification algorithms can be applied to real-world datasets to gain insights and improve predictive performance in space launch operations.

Thank you!